



American Spend

Elastic Markets Ledger Audit Report

Contents

[Elastic Markets Security Audit Report](#)

[Findings Summary](#)

[High Findings](#)

[Medium Findings](#)

[Informational Findings](#)

[Conclusion](#)

Elastic Markets Security Audit Report

Introduction

33Labs performed a security review of the Elastic Markets smart contracts. The review focused on accounting integrity, market lifecycle transitions, CLOB behavior, yield handling, promo-credit provenance, and externally exposed view and configuration surfaces.

Each finding below is populated from its complete source write-up rather than from the shortened tracking issue body.

Audit Team

Auditor	Role
Silvermist	Security Researcher
0x23r0	Security Researcher
Bobai23	Security Researcher
Favour Anigor	Security Researcher

Disclaimer

A smart contract security review cannot prove the complete absence of vulnerabilities. This review was bounded by time, available information, and the code under review. Additional reviews, a public bug bounty, operational monitoring, and careful deployment controls are recommended.

About 33Labs

33Labs is an independent smart contract security research and development group with six years of experience and over 40 completed audits. We specialize in building and auditing Solidity, Rust, Move, and DeFi systems. Learn more at 33labs.ai or follow us on X [@33labs](https://twitter.com/33labs).

Audit Scope

- **Repository:** [americanspend/markets](https://github.com/americanspend/markets)
- **Audited branch:** [audit/elastic-markets-ledger-snapshot](#)
- **Audited commit:** [4a77369d0da306b24019a0bb1592f9f3c576227d](#)
- **Total findings:** 24
- **Severity totals:** 2 High · 9 Medium · 13 Informational

Severity Definitions

Severity	Definition
High	A realistic path to material loss, insolvency, or sustained failure of a core protocol function.
Medium	A credible but constrained loss, accounting violation, or meaningful degradation of protocol behavior.
Informational	Hardening, correctness, UX, configuration, documentation, or defense-in-depth observations without material direct loss under the stated assumptions.

Findings Summary

ID	Title	Severity	Status	GitHub Issue
H-01	Yield earned on market-owned float is distributed to user depositors — a tiny depositor drains protocol yield	High	Fixed	#59
H-02	CLOB buy-reservation <code>uint128</code> downcast lets an oversized order under-reserve in the Ledger → poison bid DoSes the sell side	High	Fixed	#60
M-01	Promo laundered via bearer-token transfer — basis stays per-account, tokens move freely (root cause of the promo-laundering family)	Medium	Acknowledged / Accepted Risk	#61
M-02	Promo budget laundered into redeemable winnings via complete-set mint+burn basis dilution	Medium	Fixed	#62
M-03	CLOB self-trade wash converts promo credit into withdrawable USDC while retaining market exposure	Medium	Fixed	#55
M-04	Resolution payout-denominator is malleable after the winning outcome is revealed — promo laundered into withdrawable winnings	Medium	Fixed	#63
M-05	Complete-set burn launders promo via hidden CLOB sell-escrow basis	Medium	Fixed	#64
M-06	Seeded bids are not capped by <code>maxTick</code> — bids above the tick range strand the seed budget and provide zero Phase-2 exit liquidity	Medium	Fixed	#65
M-07	Per-level resting cap counts stale cancelled slots — a price level is cheaply, recoverably blockable	Medium	Fixed	#57
M-08	Two-wallet collusive CLOB buy routes promo spend into a controlled seller's withdrawable winnings	Medium	Acknowledged / Accepted Risk	#66
M-09	Promo budget laundered into redeemable winnings by buying both outcomes and redeeming the winner	Medium	Acknowledged / Accepted Risk	#67
I-01	Market-fund bridges are asymmetric on yield principal — <code>convertDepositToMarketHoldings</code> pays for a no-op yield sync every call	Informational	Fixed	#69
I-02	<code>MarketLens.calculatePotentialPayout</code> previews the pre-fold <code>totalPool</code> , under-reporting the resolved payout by <code>pendingSeedPoolInflow</code>	Informational	Fixed	#70

ID	Title	Severity	Status	GitHub Issue
I-03	A paused Ledger stalls graduated-market finalization — order unwinds are pause-exempt but the completion step is pause-gated	Informational	Acknowledged / Accepted Design	#71
I-04	<code>MarketLens.calculatePotentialPayout</code> divides by live outcome supply post-resolution — overquotes remaining winners as others redeem	Informational	Fixed	#73
I-05	<code>getMarketStatuses</code> / <code>getMarketDataStatus</code> report the stored <code>closed</code> flag, ignoring time-based closure — a market untradeable by time still reads <code>closed = false</code>	Informational	Fixed	#74
I-06	Cancelled-market refund via <code>redeem()</code> is pause-gated while CLOB order unwinds are pause-exempt — refunds are deferred, not blocked, during a Ledger pause	Informational	Acknowledged / Accepted Design	#75
I-07	<code>cancel()</code> has no time-gate or delay while <code>resolve()</code> enforces <code>resolutionDelay</code> — owner can cancel a market at any point, and users have no permissionless safety valve	Informational	Acknowledged / Accepted Design	#76
I-08	Cancelled-refund distribution gates the folded <code>seedFund</code> on <code>totalPool > 0</code> — a graduated market burned to <code>totalPool == 0</code> strands its retained seed backing	Informational	Fixed	#77
I-09	<code>marketBuy</code> / <code>marketSell</code> have no minimum-fill quantity floor — a partial fill succeeds silently, the only guard is the cost ceiling	Informational	Fixed	#78
I-10	<code>getMarkets</code> computes <code>offset_ + limit_</code> before clamping — a large <code>limit_</code> with any nonzero offset reverts on overflow instead of returning the tail	Informational	Fixed	#79
I-11	<code>graduateCallerRewardBps</code> is validated and reserved at creation but never paid — the entire graduate-caller bounty subsystem is dead code	Informational	Fixed	#56
I-12	Non-monotonic <code>seedFeeBpsPerTier</code> passes validation but diverges from the whole-buy runtime fee — the seed/rake safety model is misleading	Informational	Fixed	#58
I-13	<code>processFinalizationBatch</code> has no per-book fault isolation or force-skip — a single reverting order cancel would brick finalization permanently	Informational	Fixed	#80

High Findings

[H-01] Yield earned on market-owned float is distributed to user depositors — a tiny depositor drains protocol yield

Field	Value
Severity	High
Source finding	H-03
GitHub issue	#59
Affected components	<code>YieldVault._allocatableYieldSurplus / _syncYield (root cause) · YieldVault.claimYield · Ledger.deposit / _increaseYieldPrincipal</code>
Confidence	High — confirmed with a passing Foundry PoC on the dev branch
Attack class	ShareAccounting (yield over-attribution)

Description

What's wrong

The vault earns yield on every dollar it custodies — user deposits, but also market pool float (`marketHoldings`), the promo budget, treasury, and reserved buys. All of that yield is distributed only to users who hold “yield principal,” which tracks deposit + winnings balances. So a depositor earns yield not just on their own money but on everyone else's float too. A single user who deposits 1 USDC while a market holds 1,000,000 USDC of pool float collects essentially all of the yield that float generates, and withdraws it as real USDC through a permissionless, buffer-free `claimYield`.

How (mechanism)

`_allocatableYieldSurplus` (`YieldVault.sol:212-219`) is computed over the **whole** vault backing minus **all** base liabilities:

```
uint256 backing = _ledgerBacking(); // convertToAssets(balanceOf(ledger))
uint256 liabilities = _baseLiabilities() + totalAccruedYield; // incl. totalMarketHoldings, promo, treasury, ...
if (backing <= liabilities) return 0;
uint256 surplus = backing - liabilities; // = adapter yield on the FULL balance
```

`_baseLiabilities` (`YieldVault.sol:205-210`) sums `totalDeposited + totalWinnings`

- `totalPromo + totalReservedBuy + treasuryBalance + totalMarketHoldings + promoBudgetRemaining` — i.e. all non-user float is in there at face value, so the yield it earns falls entirely into surplus.

`_syncYield` (`YieldVault.sol:225-245`) then spreads that surplus across `totalYieldPrincipal`:

```
uint256 deltaIndex = Math.mulDiv(surplus, YIELD_INDEX_SCALE, principal);
accYieldPerPrincipal += deltaIndex; // principal == totalYieldPrincipal
```

But `totalYieldPrincipal` only ever increases for **user** deposit + winnings — `Ledger.deposit` → `_increaseYieldPrincipal(msg.sender, amount)` (`Ledger.sol:202`), sale/burn proceeds (`Ledger.sol:641`, `:764`, `:1245`). Market float (`depositForMarket` → `_raiseHoldings`, no principal), promo, treasury, and reserved buys earn yield but hold **zero** principal. A user's credited yield is `yieldPrincipal[user] * surplus / totalYieldPrincipal`; when they are the only principal holder, that is `surplus` in full — the yield on the entire market float.

`claimYield` (`YieldVault.sol:170-179`) is permissionless and has **no buffer check** (unlike `harvest`), so the user withdraws the credited yield as real USDC immediately.

Exploit path

1. A market holds a large pool float `F` (ordinary post-graduation state); its USDC sits in the adapter earning yield, recorded as `marketHoldings` with no yield principal.
2. Attacker `deposit(ε)` for a tiny `ε` (e.g. 1 USDC) → `yieldPrincipal[atk] = ε`, `totalYieldPrincipal = ε` (assume they are the first/only principal holder, or simply hold the dominant share).
3. The adapter accrues yield `Y` on `F + ε` ($\approx$ all of it from `F`).
4. Any op calls `_syncYield`; `surplus = Y` is spread across principal `ε`, so `claimableYield(atk)  $\approx$  Y`.
5. `claimYield()` — permissionless, no buffer check — withdraws $\approx Y$ real USDC.

Attacker contributes `ε` and extracts $\approx Y$, the yield generated by the market's float (and any promo/treasury float), which should have been harvestable protocol surplus.

Impact

- **Theft of protocol yield.** Yield generated by market pool float, promo budget, and treasury — value that `harvest` (operator, buffer-gated) would otherwise collect as protocol revenue — is siphoned by any depositor. Once `_syncYield` attributes it (raising `totalAccruedYield`, a liability), even `harvest` can no longer reach it.
- **First/dominant principal holder takes all.** With one tiny depositor the capture ratio is $\sim 100\%$; generally a holder takes `their principal / total principal` of yield earned on *everyone's* float.
- **Solvency invariant does NOT catch it.** The yield is real, so `backing  $\geq$  totalLiabilities()` holds before and after the claim — this is misappropriation of yield, not a balance-sheet hole, so the \$2.4 harness stays green.

Economic Impact: PoC — market float 1,000,000 USDC, attacker deposits **1 USDC**, adapter accrues **10,000 USDC**, attacker `claimYield()` withdraws **> 9,900 USDC** of real collateral. Loss scales with float size and adapter APY, recurring every accrual period.

Proof of Concept

`test/audit/YieldMarketFloatMisattribution.t.sol` — real `Ledger` + `YieldVault`

- mock adapter; the test contract is the registered market funding the float.

```
ledger.depositForMarket(1_000_000e6); // market float -> marketHoldings, no principal
// tiny user is the only yield-principal holder
vm.prank(tinyDepositor); ledger.deposit(1e6);
adapter.mintYield(10_000e6); // adapter yield on the full balance

assertGt(vault.claimableYield(tinyDepositor), 9_900e6); // ~all yield credited to 1-USDC depositor
vm.prank(tinyDepositor);
uint256 claimed = vault.claimYield(); // permissionless, no buffer check
assertGt(claimed, 9_900e6); // withdrew real USDC = market-float yield
```

Run output

```
$ forge test --match-contract YieldMarketFloatMisattributionTest -vv
[PASS] test_exploit_tinyDepositorCapturesMarketFloatYield() (gas: 717372)
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommendation

Yield principal must equal the cash whose yield is genuinely owed to users — exclude non-user float from the surplus, OR add an offsetting principal for it.

1. **Exclude non-user float from allocatable surplus.** Compute surplus against user-eligible backing only: `surplus = userBacking - (totalDeposited + totalWinnings)` rather than whole-vault backing minus all liabilities. Yield on `marketHoldings`, promo budget, and treasury then stays as protocol surplus and remains harvestable. This matches the actual `totalYieldPrincipal` denominator (deposit + winnings) and removes the cross-subsidy.
2. **Or track principal for every float bucket** so the index denominator covers the same dollars the surplus numerator earned on (market float yield → market, not depositors). Heavier; only needed if the protocol wants float owners to receive their own yield rather than the protocol harvesting it.

Either way, add an invariant test: yield credited to user principal must never exceed the yield earned on user-eligible backing (deposit + winnings), so yield on market/promo/treasury float can never accrue to depositors.

Related ordering quirk (separate, Low): `Ledger.deposit` calls `_syncYield()` while `balanceOf(ledger)` is still zero on the first deposit, so a pre-deposit adapter-share donation in the `totalSupply()==0` window is never parked into `unattributedYield` and instead accrues to the first principal holder. Bounded by the seed-at-deploy operational mitigation; fix by parking any positive `totalAssets()` into `unattributedYield` before the first principal is set.

Status

Fixed — The initial market-holdings correction from [PR #88](#), the yield-principal provenance work from PR #95, and the post-finalization yield routing were consolidated into merged [PR #97](#). Protocol-owned promo and seed yield now accrues to treasury, eligible principal is conserved across internal movements, and yield accruing after finalization is swept to treasury during redemption.

[H-02] CLOB buy-reservation `uint128` downcast lets an oversized order under-reserve in the Ledger → poison bid DoSes the sell side

Field	Value
Severity	High
Source finding	H-04
GitHub issue	#60
Affected components	<code>CLOB._depositFirstSpend</code> (root cause — raw <code>uint128</code> cast) · <code>CLOB._validateLimitOrderInputs</code> (no upper bound) · <code>CLOB.placeLimitBuy</code> (stores <code>uint256</code> reservation) · <code>Ledger.reserveBuyCredit</code> / <code>consumeReservedBuy</code>
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch

Field	Value
Attack class	DeltaAccounting (silent downcast / reservation mismatch → DoS)

Description

What's wrong

When a buy order rests, the CLOB records the collateral it *thinks* the order reserves as a full `uint256`, but it tells the Ledger to actually lock a value built from `uint128` fields. For an order whose notional exceeds `uint128.max`, the high bits are silently dropped: the CLOB books the order as fully backed (with a huge resting `remaining`) while the Ledger reserves only the truncated low-128-bit amount. Anyone can place such an order for ~1 USDC of real collateral. Once it rests at the best bid, market sells that cross it try to consume more reservation than the Ledger holds, revert, and the order book's sell side is frozen.

How (mechanism)

`placeLimitBuy` computes the resting notional as a `uint256` and stores it verbatim (`CLOB.sol:210-220`):

```
uint256 reservedCollateral = _getNotional(remaining, priceBps_); // uint256, unbounded
reservedCollateralByOrderId[orderId] = reservedCollateral;
ledger_.reserveBuyCredit(..., _depositFirstSpend(ledger_, msg.sender, reservedCollateral));
```

`_validateLimitOrderInputs` bounds only `amount >= minOrderSize` and the price tick — **no upper bound** (`CLOB.sol`). `_depositFirstSpend` then builds the Ledger `Basis` with `raw uint128(...)` casts — no `SafeCast`, so truncation is silent (`CLOB.sol`):

```
spend.fromDeposit = uint128(fromDeposit);
spend.fromWinnings = uint128(fromWinnings);
spend.fromPromo = uint128(amount - fromDeposit - fromWinnings); // amount == reservedCollateral
```

With `reservedCollateral = 2**128 + D` and the placer holding `D` deposit: `fromDeposit = D`, `fromPromo = uint128(2**128) = 0`. The Ledger's `reserveBuyCredit` never sees `reservedCollateral` — it reserves `_basisTotal(spend) = D` and validates that against the placer's balance (`Ledger.sol`). So the order is accepted: CLOB books `reservedCollateralByOrderId = 2**128 + D`, the Ledger locks `D`.

On a fill, `_matchIncomingSell` calls `consumeReservedBuy(orderHash, notional)` per match, and `consumeReservedBuy` reverts when `notional` exceeds the (tiny) Ledger reservation (`Ledger.sol`):

```
uint256 total = d + w + p; // == D, the truncated reservation
if (amount > total) revert ExceedsReservation();
```

Any market sell whose per-match notional exceeds `D` reverts the whole sell transaction. The CLOB order's `remaining` is huge, so it never clears and the order stays active — a permanent poison.

Exploit path

1. Attacker deposits a tiny `D` (PoC: 1 USDC).
2. `placeLimitBuy(price, amount)` with `amount` chosen so `reservedCollateral = amount.price/BPS = 2**128 + D` (PoC: `price = 5000`, `amount = (2**128 + D) * 2`). No asks → the order rests as the best bid; the Ledger locks only `D`.
3. Any seller `marketSell` s into it. `matchAmount.price/BPS > D` → `consumeReservedBuy` reverts `ExceedsReservation` → the sell transaction reverts. The best bid is frozen; every market sell crossing it fails.
4. The attacker never cancels (only they can cancel their own order), so the poison persists for ~1 USDC + gas.

Impact

- **DoS of the CLOB sell side.** A single cheap, permissionless poison bid placed at the best price freezes all market sells (and limit sells crossing that price) — core swap functionality of a graduated market is bricked.
- **Broken reservation accounting.** `reservedCollateralByOrderId` (CLOB) and the Ledger reservation diverge by up to `~2**128`; the order can never fully fill (fills past the truncated reservation revert), so it is permanently stuck.
- No direct theft — fills are gated by the Ledger reservation, so nothing is over-extracted — but availability of the order book is destroyed.

Economic Impact: PoC — attacker funds **1 USDC**, books an order the CLOB records as `2**128 + 1e6` while the Ledger reserves `1e6`. A normal 10-token market sell reverts `ExceedsReservation`. Cost to freeze the sell side: ~1 USDC.

Proof of Concept

`test/audit/CLOBReservationDowncast.t.sol` — real CLOB + Ledger + OutcomeToken.

```
ledger.deposit(1e6); // attacker funds 1 USDC
uint256 resv = (uint256(1) << 128) + 1e6; // target notional > uint128.max
uint256 amount = resv * 2; // at price 5000, notional = amount/2 = resv
uint256 id = clob.placeLimitBuy(5000, amount);

assertEq(clob.reservedCollateralByOrderId(id), resv); // CLOB: full uint256
assertEq(ledger.totalReservedBuy(), 1e6); // Ledger: truncated == D

// normal market sell into the poison bid reverts -> sell side DoSed
outcomeToken.mint(seller, 0, 10e6);
vm.prank(seller); /* setOperator */
vm.expectRevert(ILedger.ExceedsReservation.selector);
clob.marketSell(10e6, 0);
```

Run output

```
$ forge test --match-contract CLOBReservationDowncastTest -vv
[PASS] test_exploit_oversizedOrderUnderReservedThenDoSesSells()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommendation

The Ledger `Basis` buckets are `uint128` by design, so the order notional must fit `uint128`. Reject anything larger at placement rather than silently truncating:

1. **Bound the reservation** — in `placeLimitBuy` (or `_validateLimitOrderInputs`) revert when `reservedCollateral > type(uint128).max`:

```
uint256 reservedCollateral = _getNotional(remaining, priceBps_);
+ if (reservedCollateral > type(uint128).max) revert IOrderBook.AmountTooLarge();
reservedCollateralByOrderId[orderId] = reservedCollateral;
```

2. **Use checked casts** — replace the raw `uint128(...)` casts in `_depositFirstSpend` with `SafeCast.toUint128`, so any truncation reverts instead of producing a silent under-reservation. (Defense-in-depth; the bound above is the primary fix.)

Ship `test/audit/CLOBReservationDowncast.t.sol` as the regression guard — after the fix, the oversized `placeLimitBuy` must revert at placement.

Status

Fixed — The checked reservation bound and safe casts were merged in [PR #89](#). Auditor comments confirm oversized reservations now revert before placement.

Medium Findings

[M-01] Promo laundered via bearer-token transfer — basis stays per-account, tokens move freely (root cause of the promo-laundering family)

Field	Value
Severity	Medium
Source finding	M-10
GitHub issue	#61
Affected components	<code>Ledger.creditWinnings</code> (routes on the redeemer's <code>positionBasis</code>) · <code>LedgerBasisLib.routeSalePayout</code> · <code>OutcomeToken._update</code> (transfers carry no basis)
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	ShareAccounting (basis/token separation → promo provenance loss)

Description

What's wrong

Promo credit is non-withdrawable by design: when the original holder redeems a winning position, the promo principal is routed back to the non-redeemable `promoBalance` and only profit above basis becomes withdrawable. But basis is tracked per **account**, while outcome tokens are bearer instruments that move freely after graduation. Transferring promo-funded winning tokens to a fresh wallet leaves the basis behind: the transferee redeems with zero basis, so the **entire** payout — including the promo principal — books to withdrawable winnings. The promo that should never be cashable becomes cash.

How (mechanism)

`creditWinnings` routes the payout through the redeemer's `positionBasis` (`Ledger.sol:~655`):

```
PromoTypes.Basis storage b = positionBasis[user][market][outcomeId]; // the redeemer's basis
... _routeSaleProceeds(user, market, d, w, p, amount); // routeSalePayout(d,w,p,amount)
```

An `OutcomeToken` transfer touches **no** Ledger basis (no hook in `_update` — it only checks `_transfersUnlocked`). So a transferee has `positionBasis = {0,0,0}`, and `routeSalePayout(0,0,0,amount)` takes the `proceeds >= total` branch with `total == 0` → `toWinnings = amount` (all redeemable), `toPromo = 0`.

The contract documents the bearer redeem (`creditWinnings` comment / PRD §4.6: “we do not basis-gate the holder’s redeem right”). But the comment immediately below it, on `creditBurnProceeds`, documents the **opposite** intent for promo: “realized promo principal returns to `promoBalance` rather than becoming withdrawable winnings.” The two documented intents collide: bearer redeemability nullifies, with a single transfer, the promo non-withdrawability that `routeSalePayout`, `routeCompleteSetBurnPayout`, the separate `promoBalance` bucket, and `maxSubsidyPerWallet` all exist to enforce.

Exploit path

Same controller, two wallets:

1. **A** receives promo and buys winning-outcome tokens with it → `positionBasis[A] {p:X}`, A holds the tokens.
2. **A transfers** the winning tokens to a fresh wallet **B** (ERC6909 transfer, post-graduation). Basis stays with A; B holds tokens with zero basis.
3. **B redeems** → `routeSalePayout(0,0,0,payout)` → full payout to B's redeemable `winningsBalance`. B withdraws real USDC. A's `{p:X}` basis is stranded.

Sender-side corollary (same root cause, not independently exploitable)

The mirror question — *does the sender keep and reuse the stale basis for tokens someone else already consumed?* — resolves to “yes, it lingers, but it extracts nothing extra.” After A transfers the tokens, `positionBasis[A]` retains `{p:X}` even though A holds no tokens. That residue cannot be turned into value on its own:

- **Realization is bounded by A's real proceeds.** A can only realize the residue by acquiring new tokens and exiting them. On a shortfall (`proceeds < total`), `routeSalePayout` credits `toPromo = p * proceeds / total` and `creditSaleProceeds` then `delete`s the whole `positionBasis` (`Ledger.sol:672`) — A recovers promo only up to the proceeds of tokens A actually sold, never the phantom `X`.
- **Promo residue routes to `promoBalance`, not `winningsBalance`.** Even fully realized, the promo share goes back to the non-redeemable bucket — promo → promo, no laundering on the sender side.
- **On a resolved market the residue is inert.** `redeem` → `calculatePayout(A)` uses A's *held* winning tokens (zero) → `NothingToClaim`; the stale basis is never touched.
- **Accumulation dampens, never helps.** If A buys a fresh position, the residue `+=` into `total`, *lowering* the `proceeds - total` profit that would otherwise reach withdrawable winnings — an anti-exploit, not a gain.

So the sender-side residue is a correctness smell (basis outliving its tokens), not a separate extraction path. The **preferred fix below (move basis with the token) also eliminates the residue** — the sender's basis leaves with the tokens, so nothing lingers. Recorded here rather than as a standalone finding because it shares M-01's root cause and fix.

Impact

- **Loss of operator promo budget as withdrawable USDC.** Promo principal that the direct-redeem path keeps in non-redeemable `promoBalance` is converted to cash by a basis-less transferee.
- **Root cause of the family.** Every other promo-laundering finding is a more elaborate way to achieve the same basis/token separation; this is the primitive.

Economic Impact: PoC — the same 25 promo: **locked** in `promoBalance` when the basis-holder redeems (control); **fully withdrawn as USDC** when a basis-less transferee redeems (exploit).

Severity — real-world impact × likelihood

- **Impact: Medium.** The asset at risk is the operator's **promo budget** (marketing capital), not user principal or protocol solvency. Per-wallet extraction is hard-capped by `maxSubsidyPerWallet` (50 USDC). The plain bearer transfer does **not** under-back the pool — B's promo-funded winning tokens were paid into `marketHoldings`, so B's redemption is backed; it only reclassifies promo → withdrawable. So: bounded loss of promotional capital, no user-fund or solvency hit.
- **Likelihood: Medium.** `grantPromo` is `onlyRole(OPERATOR_ROLE)` — an attacker cannot self-grant. They must *receive* promo, and scaling the drain requires a **sybil against the operator's promo distribution** (many controlled wallets each granted promo). Once granted, the mechanics are trivial (transfer + redeem), but operator gating + the per-wallet cap bound real-world reachability.
- **Medium × Medium → Medium.** (A purely mechanical reading — “≤2 tx, no admin keys” — overstates this to High; “no admin keys” is technically true but the whole exploit is fed by operator-granted promo.)

Proof of Concept

`test/audit/PromoBearerTransferLaunder.t.sol` — real `Ledger` + `YieldVault` (test contract is the registered market).

```
ledger.grantPromo(A, 50e6, MANUAL);
ledger.debit(A, market, YES, Basis(0,0,25e6)); // promo-funded YES position for A
ledger.debit(A, market, NO, Basis(0,0,25e6)); // promo-funded NO position for A

// CONTROL: A redeems its own promo position -> promo principal LOCKED
ledger.creditWinnings(A, market, NO, 25e6);
assertEq(ledger.promoBalance(A), 25e6); // back to promoBalance (non-redeemable)
assertEq(ledger.winningsBalance(A), 0);

// EXPLOIT: A transfers YES tokens to fresh B (no basis moves); B redeems
ledger.creditWinnings(B, market, YES, 25e6); // routeSalePayout(0,0,0,25e6)
assertEq(ledger.winningsBalance(B), 25e6); // full payout to redeemable winnings
ledger.redeem(25e6); // B withdraws real USDC (promo budget)
```

Run output

```
$ forge test --match-contract PromoBearerTransferLaunderTest -vv
[PASS] test_exploit_bearerTransferLaundersPromoToWinnings()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommendation

Reconcile the two intents — promo principal must never become withdrawable, regardless of who holds the token. This single fix closes the whole family:

1. **Move basis with the token (preferred):** add a `Ledger` hook in `OutcomeToken._update` that transfers the proportional `positionBasis` (incl. promo) from sender to recipient on every post-graduation transfer. Then a transferee inherits the promo basis and `routeSalePayout` keeps it locked — and the mint/burn/escrow variants (M-02/M-03/M-04/M-05) lose their separation primitive too.
2. **Or** enforce the system-wide identity $\sum \text{promo-in} \geq \sum \text{promo-redeemable}$: track promo-funded supply per token id and cap redeemable winnings so promo principal can never cross to `winningsBalance` anywhere. (The consolidated class invariant — recommended as a fuzz invariant regardless of the chosen fix.)
3. **Or** (weaker) basis-gate the bearer redeem for promo-tainted tokens / make promo-funded winning tokens non-transferable.

Ship `test/audit/PromoBearerTransferLaunder.t.sol` as the regression guard, and add the $\sum \text{promo-in} \geq \sum \text{promo-redeemable}$ fuzz invariant as the class-level guard.

Status

Acknowledged / Accepted Risk — The team intentionally accepts the bearer-token promo behavior rather than moving basis with every transfer. The finding is formally acknowledged in [issue #61](#).

[M-02] Promo budget laundered into redeemable winnings via complete-set mint+burn basis dilution

Field	Value
Severity	Medium

Field	Value
Source finding	M-11
GitHub issue	#62
Affected components	<code>Ledger.creditBurnProceeds</code> (root cause) · <code>Ledger.debitForMultipleOutcomes</code> · <code>LedgerBasisLib.routeCompleteSetBurnPayout</code> · <code>Market.burnCompleteSet</code>
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	ShareAccounting / DeltaAccounting (basis-provenance reclassification)

Description

What's wrong

Promo credit is operator-funded marketing money, non-redeemable by design — only a genuine market win turns it into withdrawable cash. A user can defeat that. By holding a large balanced inventory of a market's outcome tokens, then minting a complete set funded with promo and immediately burning it, the user converts almost the entire promo amount into redeemable `winningsBalance` and withdraws real USDC — the operator's seeded promo budget — in a single round-trip, with no market risk and no resolution.

How (mechanism)

A denominator mismatch between how a complete-set mint *records* basis and how the matching burn *realizes* it:

- **Mint** — `Market.mintCompleteSet` → `Ledger.debitForMultipleOutcomes` (`Ledger.sol:457`) records the promo spend as position basis split evenly across outcomes: `setAmount/N` of `fromPromo` per outcome.
- **Burn** — `Market.burnCompleteSet` → `Ledger.creditBurnProceeds` (`Ledger.sol:575`) realizes only the burned fraction of each outcome's basis: `basis * burnedTokens / preBurnHoldings[i]` (`Ledger.sol:612`). `preBurnHoldings[i]` is the user's **entire** balance — `Market.burnCompleteSet` sets it from `_outcomeBalanceOf(msg.sender, i)` (`Market.sol:625,631`), old inventory included.
- **Routing** — burn proceeds are **fixed at face value** (`netAmount`); the profit branch of `routeCompleteSetBurnPayout` (`LedgerBasisLib.sol`) routes `proceeds - realizedBasis` to `winningsBalance` (redeemable).

When the user holds pre-existing inventory, `preBurnHoldings[i] >> burnedTokens`, so realized promo basis floors toward zero while proceeds stay at face value. The gap is booked as **profit** → `winningsBalance` — laundered promo principal. A complete-set burn is a deterministic 1:1 unwind, so no real profit occurred.

The diluting inventory is free: outcome tokens are **bearer / zero basis on transfer** (`Ledger.sol` comments at the burn/winnings paths). A second wallet mints a complete set and transfers the tokens to the attack wallet.

Exploit path (two-outcome market)

Attacker holds a large zero-basis balanced inventory `OLD` per outcome (bearer transfer from a second wallet); operator seeded the budget; attacker holds `P = 50 USDC` promo (the per-wallet cap).

1. **Mint** `50 USDC` of sets with promo → basis `{p:25}` per outcome, `marketHoldings += 50`, holdings `OLD + 50` each.
2. **Burn** the minted `50`, `preBurnHoldings = OLD + 50`. With `OLD = 100_000`: realized per outcome `≈ mulDiv(25, 50, 100050) ≈ 0`; `routeCompleteSetBurnPayout(≈0, 50)` → `toWinnings ≈ 50`.
3. **Redeem** `~50` real USDC — the operator's promo seed.

Repeat with a fresh wallet to launder another \$50.

Impact

- **Operator / protocol:** Direct theft of the seeded promo budget — ~\$1 withdrawn per \$1 of promo, capped at ~\$50/wallet but unbounded across wallets.
- **Funding-model break:** promo is supposed to convert to redeemable value only through a resolved win; this converts it with zero market exposure.
- **§2.4 solvency stays green** (`totalPromo` falls as `totalWinnings` rises), so the solvency harness does not catch it — the loss is to the promo budget.

Economic Impact: PoC launders > 49 of 50 USDC promo into redeemable winnings per wallet, withdrawn as real USDC.

Severity — real-world impact × likelihood

- **Impact: Medium.** Asset at risk is the operator's **promo budget** (marketing capital), not user principal or protocol solvency. Per-wallet take is hard-capped by `maxSubsidyPerWallet` (50 USDC). The mint+burn leaves the burned set's value paid out from `marketHoldings` it funded, so no honest-winner shortfall — pure promo-budget reclassification.
- **Likelihood: Medium.** `grantPromo` is `onlyRole(OPERATOR_ROLE)` — the attacker cannot self-grant; scaling the drain needs a **sybil against the operator's promo distribution** plus the diluting bearer inventory. Mechanics are trivial once promo is granted, but the gating + per-wallet cap bound real-world reachability.
- **Medium × Medium → Medium.** (The earlier “High, ~\$1-per-\$1, ≤2 tx” framing read the mechanics, not the real-world blast radius: bounded promo budget, operator-fed.)

Proof of Concept

`test/audit/PromoBurnLaunder.t.sol` (real `Ledger` + `YieldVault`; test contract is the registered market). `PROMO = 50 USDC == maxSubsidyPerWallet`.

```
function test_control_noOldInventory_promoStaysLocked() public {
    _seedAndGrantPromo(attacker, PROMO);
    _mintCompleteSetWithPromo(attacker, PROMO);
    _burnCompleteSet(attacker, PROMO, 0, PROMO); // no diluting inventory
    assertEq(ledger.promoBalance(attacker), PROMO); // promo round-trips back
    assertEq(ledger.winningsBalance(attacker), 0); // nothing redeemable
}

function test_exploit_oldInventoryLaundersPromoToWinnings() public {
    _seedAndGrantPromo(attacker, PROMO);
    uint256 oldInventory = 100_000e6; // heavy dilution
    _mintCompleteSetWithPromo(attacker, PROMO);
    _burnCompleteSet(attacker, PROMO, oldInventory, PROMO);
    assertGt(ledger.winningsBalance(attacker), 49e6); // promo -> redeemable
    assertLt(ledger.promoBalance(attacker), 1e6);
    vm.prank(attacker); ledger.redeem(ledger.winningsBalance(attacker)); // real USDC out
}
```

Run output

```
$ forge test --match-contract PromoBurnLaunderTest -vv
[PASS] test_control_noOldInventory_promoStaysLocked()
[PASS] test_exploit_oldInventoryLaundersPromoToWinnings()
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

The control proves intended behavior (promo round-trips back to `promoBalance`); the exploit proves the leak.

Recommendation

A pre-resolution complete-set burn is a deterministic 1:1 unwind and cannot produce profit. Realize the basis of the **set being burned**, not a slice diluted by unrelated inventory:

1. **Realize burn basis against the minted set lot** — track promo/complete-set basis in a dedicated per- `(user, market)` set-basis bucket on `mintCompleteSet` and realize it directly on `burnCompleteSet`. Complete sets are fungible, so a single set-basis bucket suffices and leaves directional-position basis intact.
2. **Or cap burn-routed winnings at genuine profit** — because a burn has no market upside, route `min(proceeds, realizedBasis)` through the basis buckets and forbid the `proceeds > total` → winnings branch for the burn path.

Either fix must keep the control test passing (fresh-user mint+burn returns promo to `promoBalance`) while making the exploit credit the laundered amount back to `promoBalance`. Ship `test/audit/PromoBurnLaunder.t.sol` as the regression guard.

Status

Fixed — The complete-set basis correction and regression tests were consolidated from PR #96 into merged [PR #97](#). Auditor review confirms complete-set mint and burn no longer dilute promo basis.

[M-03] CLOB self-trade wash converts promo credit into withdrawable USDC while retaining market exposure

Field	Value
Severity	Medium
Source finding	M-12
GitHub issue	#55
Affected components	<code>CLOB._matchIncomingSell</code> (root cause — no self-match guard + independent two-leg settlement) · <code>Ledger.consumeReservedBuy</code> · <code>Ledger.consumeEscrowedSellBasis</code> / <code>_routeSaleProceeds</code>
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	MEVEconomic / DeltaAccounting (wash-trade self-cross, basis double-count)

Description

What's wrong

The order book never checks whether the resting maker and the incoming taker are the same address. A user places a promo-funded bid and market-sells their own outcome tokens into it. The Ledger settles the two legs independently, so the same user receives the buyer's outcome (promo recorded as position basis) **and** the seller's proceeds (booked as redeemable `depositedBalance` / `winningsBalance`). The proceeds are funded by the user's own promo bid, so non-redeemable promo turns into withdrawable USDC — and because the user is also the maker, the tokens come straight back, leaving their market exposure unchanged.

How (mechanism)

`_matchIncomingSell` (`CLOB.sol:943-1061`) has no `maker == taker` guard (the `makerOrder.trader == market` checks at `:879/:997/:1033` are for the seeded-bid burn, a different case) and settles each crossed bid in two independent Ledger calls, then returns the tokens to the maker:

```

ILedger(ledger).consumeReservedBuy(_orderHash(makerOrderId), notional); // :989 promo -> position basis +
marketHoldings
_consumeTakerSellEscrow(taker, takerOrderHash, matchAmount, notional, takerFee); // :992 realize basis, credit proceeds
...
_safeTransferOutcome(makerOrder.trader, matchAmount); // :1001 maker == taker -> tokens come back

```

- `consumeReservedBuy` moves the reserved **promo** into the maker's `positionBasis.fromPromo` for the full `notional` and does `marketHoldings += notional`.
- `_consumeTakerSellEscrow` → `consumeEscrowedSellBasis` → `_routeSaleProceeds` routes `proceeds = notional - takerFee` against the seller's realized basis; the profit branch (`proceeds ≥ basis`) credits the excess to `winningsBalance`; `marketHoldings -= proceeds`.

A promo-funded bid is reachable: `placeLimitBuy` funds via `_depositFirstSpend` (`CLOB.sol:242`) — with deposit/winnings exhausted it returns `{fromPromo: notional}`. Bid price is user-chosen within `[minTick, maxTick]`. `marketHoldings` nets to zero (`+notional` then `-proceeds`), but the user's claims do not: the buy leg created `notional` of promo position basis **and** the sell leg paid out `notional` of proceeds. The same promo is counted twice; the difference `proceeds - basis` is fabricated redeemable winnings, and the leftover promo position is left under-backed.

Exploit path

Single outcome, attacker holds `q` tokens at cost basis `b/token` (`b < p`), bid price `p`:

1. Hold `q` low-basis tokens; spend down deposit/winnings so only promo remains.
2. `placeLimitBuy(p, q)` — funds from promo, `notional = p·q` rests as a bid.
3. `marketSell(q, 0)` — sweeps the own bid:
 - `consumeReservedBuy(notional)` → `positionBasis.fromPromo += p·q`, `marketHoldings += p·q`.
 - `consumeEscrowedSellBasis` → `depositedBalance += b·q`, `winningsBalance += (p-b)·q`, `marketHoldings -= p·q`.
 - maker (attacker) receives the `q` tokens back — exposure unchanged.
4. `redeem((p-b)·q + b·q)` — withdraws real USDC; the `(p-b)·q` is laundered promo.

Claims created: position `p·q` (promo) + proceeds `p·q` - promo spent `p·q` - basis consumed `b·q` = `(p-b)·q` **fabricated**.

Impact

- **Theft of operator promo budget** → **withdrawable USDC**, keeping market exposure; bounded ~\$40–45/wallet, repeatable across wallets.
- **Market insolvency seed**: each wash leaves an under-backed promo position (`marketHoldings` net-zeroed while `positionBasis.fromPromo` claims the full notional); on a winning resolution those tokens are owed payouts the market cannot cover (`InsufficientMarketHoldings`), shifting the shortfall to others.
- **§2.4 solvency harness misses it** (promo → winnings reclassification within liabilities).

Economic Impact: PoC — attacker puts in **\$5** own capital, withdraws **~\$45**, retains 50 tokens: ~\$40 promo extracted per cycle, ~\$40 unbacked position left.

Severity — real-world impact × likelihood

- **Impact: Medium (escalation factor noted)**. The directly-extracted asset is the operator's **promo budget** (~\$40/cycle), per-wallet capped by `maxSubsidyPerWallet` (50 USDC). Escalation: the self-cross leaves an **under-backed position** that can short honest winners at resolution (real user-fund impact) — but bounded by the promo extracted, so it nudges toward Medium-High at most, not clean High.

- **Likelihood: Medium.** `grantPromo` is `onlyRole(OPERATOR_ROLE)` — the attacker cannot self-grant; scaling requires a **sybil against the operator's promo distribution**, and the wash needs a bid that rests above the seller's basis (thin ask side / appreciated holdings). Trivial once those hold.
- **Medium × Medium → Medium.** (Earlier “High, ≤2 tx, no admin keys” over-weighted the mechanics over the operator-gated, per-wallet-capped real-world reach.)

Proof of Concept

`test/audit/PromoSelfTradeWash.t.sol` — real CLOB + Ledger; the test contract stands in for the Market lifecycle surface (`graduated()==true` opens trading and unlocks ERC6909 transfers). Promo = 50 USDC (`maxSubsidyPerWallet`).

```
outcomeToken.mint(attacker, ID, 50e6);
ledger.debit(attacker, market, ID, Basis(5e6, 0, 0)); // $0.10/token basis; empties deposit
ledger.grantPromo(attacker, 50e6, MANUAL);

vm.startPrank(attacker);
clob.placeLimitBuy(9000, 50e6); // $0.90 bid, 45 USDC promo reserved
outcomeToken.setOperator(address(clob), true);
clob.marketSell(50e6, 0); // sell into own bid
vm.stopPrank();

assertGt(winningsBalance(attacker), 39e6); // ~40 USDC promo -> winnings
assertEq(outcomeToken.balanceOf(attacker, ID), 50e6); // exposure retained
ledger.redeem(45e6); // withdraws real USDC
assertLt(marketHoldings(market), positionPromo); // position under-backed
```

Run output

```
$ forge test --match-contract PromoSelfTradeWashTest -vv
[PASS] test_exploit_selfTradeLaundersPromoToWinnings()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommendation

Prevent self-matching — the minimal, standard CLOB fix that kills the double-count at the source. In

`_matchIncomingBuy` / `_matchIncomingSell`, skip (or revert on) a maker order whose `trader == taker`:

```
uint256 makerOrderId = queue[head];
IOrderBook.Order storage makerOrder = orders[makerOrderId];
if (!makerOrder.active || makerOrder.remaining == 0) { head++; continue; }
+ // A trader cannot cross their own resting order: settling both legs against
+ // one actor mints redeemable winnings out of their own (promo-funded) bid.
+ if (makerOrder.trader == taker) { head++; continue; }
```

Skipping lets the taker continue matching against other makers. Defense-in-depth: settle a detected self-cross as a net no-op (return tokens, release the reservation, book no proceeds/winnings). After the fix, the self-cross is skipped and `marketSell` against only-own-bids reverts `InsufficientLiquidity` — ship `test/audit/PromoSelfTradeWash.t.sol` as the regression guard.

Status

Fixed — Self-cross rejection for both matching directions and the promo-wash regression tests were consolidated into merged [PR #97](#).

[M-04] Resolution payout-denominator is malleable after the winning outcome is revealed — promo laundered into withdrawable winnings

Field	Value
Severity	Medium
Source finding	M-13
GitHub issue	#63
Affected components	<code>Market.resolve / _startPendingFinalization / _finalizeResolvedMarket</code> (root cause — deferred snapshot) · <code>Market._externalOutcomeSupply · OutcomeToken._update</code> (transfers unlocked while graduated) · <code>Market.calculatePayout / redeem → Ledger.creditWinnings</code>
Confidence	High — confirmed with a passing integration PoC on the <code>dev</code> branch
Attack class	ShareAccounting (payout-denominator manipulation → promo → redeemable laundering)

Description

What's wrong

For a graduated market, calling `resolve()` reveals the winning outcome but does **not** immediately fix the winning-token supply used to split the payout. That supply is snapshotted later, in a separate finalization transaction. Between the two, outcome-token burns and transfers are still allowed, so a holder who already knows the winning outcome can shrink the supply denominator while the payout pool stays full — inflating every remaining winner's redeemable payout. A colluding promo wallet burns its winning tokens (whose payout would otherwise route to non-redeemable `promoBalance`), shifting that share to a cash wallet where it lands in redeemable `winningsBalance`. Promo-budget USDC becomes withdrawable.

How (mechanism)

`resolve(correctOutcome_)` for a graduated market takes the deferred path (`Market.sol:363-367`):

```
if (graduated) {
  _startPendingFinalization(RESOLVE, correctOutcome_, false); // stores + EMITS the outcome
  return; // <-- returns WITHOUT snapshotting
}
_finalizeResolvedMarket(correctOutcome_); // non-graduated path snapshots atomically
```

`_startPendingFinalization` emits `FinalizationStarted(type, correctOutcome_, ...)` (`Market.sol:1277`) — the winning outcome is now public. The supply snapshot is taken only later, when `processMarketFinalizationBatch` finishes and calls `_finalizeResolvedMarket` (`Market.sol:1287-1289`):

```
winningTokenSupplyAtResolution = _externalOutcomeSupply(correctOutcome_); // totalSupply - market balance
```

These are **separate transactions** (`resolve` is `onlyFactory` and returns; `processMarketFinalizationBatch` is a distinct `onlyFactory` call). During the gap the market is `closed` and `graduated` but not `resolved`, and `OutcomeToken._update` gates transfers only on `_transfersUnlocked = graduated || resolved` (`OutcomeToken.sol:215-228`) → **transfers and burns are unlocked**. `OutcomeToken.burn(id, amount)` and a transfer to `address(market)` (subtracted by `_externalOutcomeSupply`) both reduce the denominator.

`payoutPool = totalPoolValue - rake (Market.sol:1328)` is independent of the winning supply, and `calculatePayout` is purely token-based (`Market.sol:967-972`):

```
return (userWinningTokens * payoutPool) / winningTokenSupplyAtResolution;
```

So shrinking `winningTokenSupplyAtResolution` after `resolve` reveals the outcome inflates each remaining token's payout. `redeem` routes the inflated payout through `Ledger.creditWinnings` (`Market.sol:418-421`), which respects basis provenance — so the inflated share landing on a **cash** position becomes redeemable `winningsBalance`, while the burned **promo** position's share (which would have routed to non-redeemable `promoBalance`) is simply forfeited.

Exploit path

Same controller, two wallets, both holding the eventual winning outcome:

- **A** — promo-funded winning tokens (`positionBasis {p:Pa}`).
 - **B** — cash-funded winning tokens (`positionBasis {d:Db}`).
1. Factory `resolveMarket(market, win)` → graduated path → pending finalization; `FinalizationStarted` reveals `win`. No snapshot yet.
 2. Attack window: **A burns** its winning tokens (`OutcomeToken.burn`) — allowed, market still graduated. `_externalOutcomeSupply` drops by A's holding.
 3. Factory `processMarketFinalizationBatch` → `_finalizeResolvedMarket` snapshots the **reduced** denominator.
 4. **B redeems** → `calculatePayout(B) = Nb · payoutPool / (reduced supply)` — inflated. `creditWinnings` books it to redeemable `winningsBalance`.

A's promo position is sacrificed (it was non-redeemable anyway); its payout share, backed by the promo budget that sits in `marketHoldings`, flows to B as withdrawable USDC.

Impact

- **Direct theft of promo budget → withdrawable USDC.** Promo-funded winning value that should route to non-redeemable `promoBalance` is laundered into B's redeemable `winningsBalance`. Same family as [M-02]/[M-03]; bounded per wallet by `maxSubsidyPerWallet`, repeatable across wallets.
- **Payout-integrity break (independent of promo).** The winning-supply denominator is malleable after the outcome is public — a colluding pair can redistribute the payout pool; honest co-winners are diluted relative to the snapshot the protocol intended (the supply at resolution, not at finalization).

Economic Impact: PoC — two identical markets, A (promo 1000) + B (cash 1000) both on the winning outcome:

- Honest finalize: **B redeems 857 USDC** (deposit-basis recovery, 0 winnings); A's 857 stays locked in `promoBalance`.
- A burns before the snapshot: **B redeems 2000 USDC** — the *entire* `payoutPool` — of which **1000 is redeemable winnings**.
- Colluder put in **1000 cash**, withdrew **2000** → **~1000 USDC of promo budget extracted as profit**, funded by A's forfeited (promo-backed) share.

Severity — real-world impact × likelihood

- **Impact: Medium.** Extracted asset is the operator's **promo budget**, per-wallet capped by `maxSubsidyPerWallet` (50 USDC) — the PoC's 1000 figure assumes a raised cap; at production caps the per-cycle take is ~\$50. The redistribution shorts other winners' share of `payoutPool` (real user-fund impact) but bounded by the forfeited promo. No protocol-solvency hole (the pool stays fully paid out).
- **Likelihood: Medium.** Requires `OPERATOR_ROLE`-granted promo (no self-grant; scale needs a **sybil** across promo distribution), a graduated market, and acting in the post-`resolve`/pre-snapshot window (front-running `processMarketFinalizationBatch`). Mechanically reliable but multi-step and operator-fed.

- **Medium × Medium → Medium.** (Earlier “High, ≤2 tx, no admin keys” overstated it: bounded promo budget, operator-gated, multi-step timing.)

Proof of Concept

`test/audit/PromoResolutionDenominatorWash.t.sol` — full `MarketFactory` + `Ledger`

- `Market` lifecycle (extends the `EndToEnd` harness). Runs two identical markets; only the second burns A's winning tokens during the post-`resolve` pending window.

```
// graduated market, A (promo) + B (cash) both hold winning outcome 0
factory.closeMarket(address(m2));
factory.resolveMarket(address(m2), 0); // graduated -> PENDING; outcome 0 revealed; NOT snapshotted
vm.prank(A); ot2.burn(0, aTokens);      // attack window: burn promo-funded winning tokens
_processAllPendingFinalization(address(m2)); // snapshot now excludes A's burned tokens
vm.prank(B); m2.redeem();                // B captures the whole payoutPool as redeemable

assertGt(bManipulated, bHonest + 100e6); // B's redeemable inflated by the promo share
```

Run output

```
$ forge test --match-contract PromoResolutionDenominatorWashTest -vv
[PASS] test_exploit_burnWinningSupplyBeforeSnapshotInflatesPayout()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Trace: honest `Redeemed(B, 857191839)`; manipulated `Redeemed(B, 2000000000)` (`= payoutPool`), with `routeSalePayout({d:1000e6}, 2000e6) → 1000e6 winnings`.

Recommendation

Snapshot the winning supply when the outcome is decided, not when finalization finishes — and/or freeze token mutation during finalization. Either closes the gap:

1. **Snapshot at `resolve` / finalization start.** Capture `winningTokenSupplyAtResolution = _externalOutcomeSupply(correctOutcome_)` inside `_startPendingFinalization` (the moment the outcome becomes known), not in the deferred `_finalizeResolvedMarket`. The batched book cleanup can still run later, but the denominator is fixed before anyone can react to the revealed outcome.
2. **Block burns/transfers during finalization.** Extend `OutcomeToken._update` to revert while `pendingFinalizationType != NONE` (treat pending finalization like a locked window), so the supply cannot move between `resolve` and the snapshot.

Option 1 is the targeted fix (the deferral is the root cause; the non-graduated path already snapshots atomically). Ship `test/audit/PromoResolutionDenominatorWash.t.sol` as the regression guard — after the fix, burning A's tokens post-`resolve` must not change B's payout.

Status

Fixed — The pending-finalization supply protections and internally tracked settlement denominator were consolidated into merged [PR #97](#). Auditor review confirms the resolution denominator can no longer be changed after the winning outcome is revealed.

[M-05] Complete-set burn launders promo via hidden CLOB sell-escrow basis

Field	Value
Severity	Medium
Source finding	M-14
GitHub issue	#64
Affected components	<code>Ledger.creditBurnProceeds</code> (realizes <code>positionBasis</code> only) · <code>Ledger.escrowSellBasis</code> (moves basis out of <code>positionBasis</code>) · <code>Ledger.releaseEscrowedSellBasis</code> (restores it) · <code>LedgerBasisLib.routeCompleteSetBurnPayout</code>
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	ShareAccounting / DeltaAccounting (basis-provenance reclassification)

Description

What's wrong

Promo credit is non-redeemable by design. A user can convert it to withdrawable USDC by hiding an outcome's promo basis inside a resting CLOB sell order while burning a complete set. The burn pays face value but realizes a reduced basis (the hidden portion is invisible to the burn), so the gap is booked as redeemable winnings. The user then cancels the resting sell to restore the hidden promo basis and tokens — leaving the promo “conserved” on paper while real redeemable winnings have been minted from the promo budget.

How (mechanism)

`escrowSellBasis` moves an outcome's basis out of `positionBasis` into sell escrow (`Ledger.sol:958` : `pos.fromPromo = (p - ep)`), and `releaseEscrowedSellBasis` restores it on cancel (`Ledger.sol:1270-1272`). But `creditBurnProceeds` realizes basis **only from** `positionBasis` (`Ledger.sol:595-617`) — it never consults the sell escrow. So basis parked in a resting sell is invisible to the burn.

Because a complete-set burn pays **face value** (1:1, no appreciation), `routeCompleteSetBurnPayout` routes `proceeds - realizedBasis` to redeemable `winningsBalance` whenever the realized basis is below face value. Hiding promo basis in escrow drives the realized basis down → the gap becomes phantom winnings.

Exploit path (binary market)

1. **Mint** a complete set with promo → `positionBasis {YES:p25, NO:p25}`, hold 50 YES + 50 NO. (`debitForMultipleOutcomes`)
2. **Place a non-filling YES sell** → `escrowSellBasis` moves YES promo basis (25) into escrow; `positionBasis[YES]` becomes empty; the 50 YES tokens go to the CLOB.
3. **Acquire replacement YES** with zero/low basis (bearer transfer from a second wallet, or buy below the hidden promo price) → the wallet holds a full complete set again, but `positionBasis[YES]` carries only the cheap replacement basis.
4. **Burn the complete set** (`creditBurnProceeds`, `preBurnHoldings = 50` each): YES realizes 0 (hidden), NO realizes p25 → `routeCompleteSetBurnPayout({0,0,25}, 50)` → **promo 25, winnings 25**. The 25 winnings is redeemable.
5. **Cancel the resting YES sell** → `releaseEscrowedSellBasis` restores `positionBasis[YES] = {p:25}` and returns the original 50 YES tokens.

Promo is conserved on paper ($25 \text{ promoBalance} + 25 \text{ restored position} = 50$), yet 25 of redeemable winnings was minted from the promo budget. The restored YES tokens are left under-backed (the burn drained the pool) — the insolvency seed shared with [M-02]/[M-03]/[M-04].

Relationship to M-02 — do they share a root cause?

Same abstract root cause; different basis-reduction vector; same cleanest fix.

- **Shared root cause:** a complete-set burn pays **face value (1:1)** while `creditBurnProceeds` realizes a *reducible* basis, so any reduction below face value mints phantom redeemable winnings. Both findings exploit exactly this.
- **Different vector for reducing the realized basis:**
 - **M-02** inflates the realization *denominator* — `basis * burnedTokens / preBurnHoldings` — by padding the wallet with unrelated zero-basis inventory, flooring the realized promo basis. The promo basis stays in `positionBasis`, under-realized.
 - **M-05** removes basis from `positionBasis` entirely by relocating it to sell escrow (`escrowSellBasis`), which `creditBurnProceeds` cannot see, then restores it via cancel.
- **Fix-distinctness (per the bundling discipline — share a submission only if root cause AND remediation match):**
 - A **narrow** vector-specific patch does NOT close both: fixing M-02's denominator (realize against the minted set, not `preBurnHoldings`) leaves M-05's escrow path open, and vice-versa. At that granularity they are **fix-distinct** → **separate submissions**.
 - The **shared/cleanest** fix DOES close both: forbidding the profit branch on a 1:1 complete-set burn (route `min(proceeds, realizedBasis)`; never send `proceeds - basis` to winnings), OR realizing against a dedicated minted-set lot that neither wallet-inventory nor escrow can perturb. At that granularity they are **one fix** → **one submission**.

So: treat M-05 as a **variant of M-02** (same class, same recommended invariant), but document it separately because the *escrow* basis-leak site is distinct and a vector-specific patch to M-02 alone would not fix it.

Impact

- **Theft of operator promo budget** → **withdrawable USDC**, repeatable across wallets (bounded per wallet by `maxSubsidyPerWallet`, ~\$25–50/cycle).
- **Under-backed retained position:** the restored outcome tokens claim basis the pool no longer backs, contributing to market insolvency at resolution.
- **§2.4 solvency harness misses it** (promo → winnings reclassification within liabilities), same as the other promo-laundering findings.

Economic Impact: PoC — 50 promo grant yields **25 redeemable winnings** withdrawn as real USDC per cycle, with the promo basis fully restored for reuse.

Severity — real-world impact × likelihood

- **Impact: Medium (escalation factor noted).** Directly extracts the operator's **promo budget** (~\$25/cycle, one outcome's promo hidden per cycle), per-wallet capped by `maxSubsidyPerWallet`. Escalation: the restored tokens are left **under-backed** (burn drained the pool), which can short honest winners at resolution — bounded by the promo extracted.
- **Likelihood: Medium.** `OPERATOR_ROLE`-granted promo (no self-grant; scale needs a **sybil**), plus a zero/low-basis replacement token and a resting non-filling sell. Multi-step but mechanically reliable.
- **Medium × Medium** → **Medium.** (Earlier “High” read the mechanics; the real-world blast radius is bounded promo budget, operator-fed.)

Proof of Concept

`test/audit/PromoBurnEscrowHide.t.sol` (real `Ledger` + `YieldVault` ; test contract is the registered market, feeding the escrow/burn calls the CLOB/Market make).

```
_mintCompleteSetWithPromo(attacker, 50e6); // {YES:p25, NO:p25}
ledger.escrowSellBasis(attacker, market, YES, sellHash, 50e6, 50e6); // hide YES p25
// replacement YES is zero-basis (bearer) -> positionBasis[YES] stays empty
_burn(attacker, 50e6, 50e6); // YES realizes 0, NO realizes p25
assertEq(ledger.winningsBalance(attacker), 25e6); // redeemable minted from promo
assertEq(ledger.promoBalance(attacker), 25e6);
ledger.releaseEscrowedSellBasis(attacker, sellHash); // restore hidden YES p25
(, , uint128 yesPromo) = ledger.positionBasis(attacker, market, YES);
assertEq(yesPromo, 25e6); // promo "conserved" on paper
ledger.redeem(25e6); // withdraw promo-funded USDC
```

Run output

```
$ forge test --match-contract PromoBurnEscrowHideTest -vv
[PASS] test_control_honestBurn_noWinnings() // honest burn: 0 winnings, all promo restored
[PASS] test_exploit_escrowHidesPromoBasisFromBurn() // trick: 25 redeemable winnings minted
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

The control proves the intended behaviour (no escrow → 0 winnings, all promo to `promoBalance`); the exploit proves the leak.

Recommendation

Adopt the shared invariant that closes the whole class (and both M-02 and M-05):

1. **Realize complete-set burn basis against a dedicated minted-set lot**, not pro-rata over `preBurnHoldings`, so neither padded wallet inventory nor parked sell-escrow basis can change what the burn realizes. (Preferred — single fix.)
2. **Or forbid the profit branch on the 1:1 burn unwind** in `routeCompleteSetBurnPayout` : route `min(proceeds, realizedBasis)` and never credit `proceeds - basis` to `winningsBalance` for a complete-set burn.
3. **Escrow-specific belt-and-suspenders**: in `creditBurnProceeds`, fold any of the user's `escrowedSellBasis` for the burned outcomes back into the realized basis, or block `burnCompleteSet` while the user has resting sell escrow on any outcome of the set.

Ship `test/audit/PromoBurnEscrowHide.t.sol` (alongside `PromoBurnLaunder.t.sol`) as the regression guard for the class.

Status

Fixed — The burn and active-sell-escrow basis protections were consolidated into merged [PR #97](#). Auditor review confirms complete-set burns can no longer hide promo basis in sell escrow.

[M-06] Seeded bids are not capped by `maxTick` — bids above the tick range strand the seed budget and provide zero Phase-2 exit liquidity

Field	Value
Severity	Medium
Source finding	M-15

Field	Value
GitHub issue	#65
Affected components	<code>CLOB.seedLiquidity</code> (no tick validation) · <code>CLOB._computeBidLadder</code> (no <code>maxTick</code> clamp) · <code>MarketConstants.seedOutcomeBook</code> / <code>Market._seedOrderBookLiquidity</code> (unclamped implied price) · <code>CLOB._prevActiveBuyPrice</code> / <code>getTopOfBook</code> / <code>marketSell</code> (scan clamps to <code>maxTick</code>)
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	TickBoundaryMath (price-range mismatch → stranded liquidity)

Description

What's wrong

When a market graduates, the protocol seeds the order book with bids that are supposed to give holders exit liquidity. Those seeded bids are inserted with no upper price bound, while every path that *reads* or *matches* the book caps its scan at `maxTick`. So if the seed price exceeds `maxTick`, the seeded bids land above the searchable range: they sit active in storage and lock up the market's seed budget, but no `marketSell` can ever reach them. The seeding does nothing for the affected outcomes — holders have no exit liquidity — even though the budget was reserved for it.

How (mechanism)

The insertion path applies no `maxTick` check, and the ladder math never clamps:

- `CLOB.seedLiquidity` (`CLOB.sol:492-530`) inserts each bid directly from `bidPrices[i]` — no validation, unlike `placeLimitBuy` → `_validateLimitOrderInputs` which rejects `priceBps > maxTick`.
- `CLOB._computeBidLadder` builds bid prices as `impliedPriceBps - (k+1)*gap` descending toward `minTick`. It receives `minTick` but **not** `maxTick` — only the ask ladder (`_computeAskLadder`) is passed `maxTick`. So the top bid is `impliedPriceBps - gap` with no upper clamp.
- `MarketConstants.seedOutcomeBook` passes `impliedPriceBps` straight into `buildSeedLadders`. That value is `seedPriceBps = _getBuyImpliedOdds(i)` (`Market.sol:1638-1643`), which ranges up to `~BPS_DENOMINATOR` (10_000 bps) — it is never clamped to `maxTick`.

Every read/match path, by contrast, clamps to `maxTick`:

- `getTopOfBook` best bid = `_prevActiveBuyPrice(maxTick, minTick)` (`CLOB.sol:707`).
- `marketSell` → `_matchIncomingSell` starts at `_prevActiveBuyPrice(maxTick, ...)` (`CLOB.sol:951`), and `_prevActiveBuyPrice` clamps from to `maxTick` (`CLOB.sol:1219`).

So when `impliedPriceBps - gap > maxTick`, the top seeded bids are stored above the scan ceiling and are permanently invisible to live trading.

Reachability

`seedPriceBps = _getBuyImpliedOdds(i)` exceeds `maxTick` whenever:

- an admin sets `maxTick` below the natural odds range for the market, or
- an outcome is imbalanced enough that its implied odds exceed `maxTick` (a dominant outcome's odds approach 10_000 bps).

`maxTick` is a per-market admin parameter with no enforced relationship to the seed implied price, so the precondition is reachable through ordinary market configuration — not an adversarial-admin scenario.

Impact

- **Seeded exit liquidity silently fails.** For any outcome whose seed implied price exceeds `maxTick`, the seeded bids are unreachable by `marketSell` — the market provides no Phase-2 exit liquidity for holders of those outcomes, which is the entire purpose of bid seeding.
- **Seed budget stranded during trading.** The market's seed budget is reserved against the unreachable bids (`reserveBuyCredit` locks `depositedBalance[market]`), so it cannot back any *reachable* liquidity either. It is recovered only at finalization (bids in `(maxTick, 10_000]` are reachable by the from- `10_000` finalization scan) — so the loss is stranded-during-trading, not permanent, *unless* `maxTick > 10_000` (then it strands permanently — see the cross-reference above).
- No direct theft; the harm is a broken core guarantee (exit liquidity) and mis-deployed seed capital. Medium: a protocol-availability failure for affected outcomes, reachable through normal admin configuration.

Proof of Concept

`test/audit/SeededBidAboveMaxTick.t.sol` — real `CLOB` + `Ledger` + `OutcomeToken`, `maxTick = 5000`.

```
// seedLiquidity accepts a bid at price 8000 > maxTick (5000) WITHOUT reverting
clob.seedLiquidity(/* bidPrices */ [8000], /* bidAmounts */ [10e6], [], []);

IOrderBook.OrderData memory o = clob.getOrder(1);
assertTrue(o.active); assertEquals(o.priceBps, 8000);           // active in storage, above maxTick
assertEq(ledger.totalReservedBuy(), 8e6);                       // seed budget reserved against it

(uint256 bestBid, uint256 bidSize,,) = clob.getTopOfBook();
assertEq(bestBid, 0); assertEquals(bidSize, 0);                // invisible to top-of-book

vm.expectRevert(IOrderBook.InsufficientLiquidity.selector);
clob.marketSell(5e6, 0);                                       // unreachable -> no exit liquidity

// symmetry: a USER order at the same price IS rejected
vm.expectRevert(IOrderBook.InvalidPriceTick.selector);
clob.placeLimitBuy(8000, 10e6);                                // seedLiquidity bypasses this guard
```

Run output

```
$ forge test --match-contract SeededBidAboveMaxTickTest -vv
[PASS] test_exploit_seededBidAboveMaxTickIsUnreachable()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommendation

Clamp the seed implied price to the tradeable range before building the ladders, so no seeded bid can ever exceed `maxTick`:

```
// Market._seedOrderBookLiquidity (before _seedOutcomeBook):
uint256 seedPriceBps = _getBuyImpliedOdds(i);
if (seedPriceBps > cfg.maxTick) seedPriceBps = cfg.maxTick;
if (seedPriceBps < cfg.minTick) { /* skip seeding this outcome */ }
```

and/or pass `maxTick_` into `_computeBidLadder` (symmetric with the ask ladder) so the bid ladder is bounded on both sides. Belt-and-suspenders: add a `priceBps <= maxTick` (and `>= minTick`) check inside `seedLiquidity` itself, so the insertion primitive can never store an out-of-range bid regardless of caller — this also closes the permanent-strand compound with the `maxTick > 10_000` finalization finding. Ship `test/audit/SeededBidAboveMaxTick.t.sol` as the regression guard.

Status

Fixed — Seeded bids are constrained to the configured tick range and validated at insertion. The correction was merged in [PR #89](#).

[M-07] Per-level resting cap counts stale cancelled slots — a price level is cheaply, recoverably blockable

Field	Value
Severity	Medium
Source finding	M-16
GitHub issue	#57
Affected components	<code>CLOB.placeLimitBuy</code> (cap uses <code>length - head</code>) · <code>CLOB._cancelActiveOrder</code> (marks inactive, never pops) · <code>CLOB.compactBuyQueue</code> / <code>_compactQueueHeadAfterCancel</code> (front-only head advance) · the unused, correct <code>activeBuyCountByPrice</code>
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	AccessControl / DoS (off-by-counter defeats a resting-depth security control)

Description

What's wrong

The order book caps each price level at 256 resting orders to bound matching/ finalization scans. But the cap counts *cancelled* slots as if they were live. Cancelling an order frees its collateral and marks it inactive, yet its slot stays in the queue and keeps counting toward the cap as long as a live order sits in front of it. So an attacker can keep one live order at the front of a level and cancel the other 255 — the level reads as “full” (256) while only one order is active. Legitimate makers can no longer post resting buys at that price, and queue compaction cannot fix it because the front order is still live. The attacker recovers ~127.5 of the ~128 USDC used, leaving the level blocked for ~0.5 USDC.

How (mechanism)

The cap check in `placeLimitBuy` uses the raw FIFO window, not the maintained active counter (`CLOB.sol:224-230`):

```
uint256[] storage buyQueue = buyOrderIdsByPrice[priceBps_];
if (buyQueue.length - buyQueueHeadByPrice[priceBps_] >= MAX_RESTING_ORDERS_PER_LEVEL) {
    revert IOrderBook.OrderBookLevelFull();
}
```

The protocol already maintains `activeBuyCountByPrice` (`CLOB.sol:129`), incremented in `_incrementActiveBuyLevel` on place/seed and decremented in `_decrementActiveBuyLevel` on fill/cancel/dust-cleanup — i.e. the exact live count the cap should use — but the cap ignores it.

`_cancelActiveOrder` (`CLOB.sol:1148+`) sets `order.active = false`, decrements `activeBuyCountByPrice`, refunds collateral, but **never pops the queue slot**. The head only advances past **leading** stale ids: both `_compactQueueHeadAfterCancel` and `compactBuyQueue` break at the first live order (`if (o.active && o.remaining != 0) break;`, `CLOB.sol:464`). So a stale slot behind a live front order is permanent dead weight in `length - head`, while `activeBuyCountByPrice` correctly drops.

Exploit path

For any target price level `P`:

1. Place 256 minimum-size buy orders at `P` → `length = 256`, `head = 0`, `activeBuyCountByPrice = 256`. Level full. (~128 USDC reserved at 0.5 USDC each.)
2. Cancel orders #1...#255, **keeping order #0 (the front) live**. Each cancel refunds collateral and decrements `activeBuyCountByPrice` → 1, but leaves the slot. `_compactQueueHeadAfterCancel` cannot advance the head (front #0 is live).
3. Result: `length - head = 256` (cap full) while `activeBuyCountByPrice = 1`. `placeLimitBuy(P, ...)` reverts `OrderBookLevelFull` for everyone; `compactBuyQueue(P, ...)` cannot free it. ~127.5 USDC recovered; ~0.5 USDC locked.
4. **Maintain** the block as the front advances by re-placing one order per fill/compaction step, pinning `length - head` at 256 indefinitely.

Repeat per level to suppress a whole band of bid prices around the mid.

Impact

- **DoS of the buy side at chosen price levels.** Legitimate makers cannot post resting bids at any blocked level — the per-level cap (added as a security control for dead-run DoS) is turned against honest users.
- **Cheap, recoverable, repeatable.** ~0.5 USDC of locked capital per level (the rest refunded); repeatable across levels to blanket the competitive bid band, harming price discovery and forcing sellers to accept worse fills.
- No fund theft and funds are not locked (fully recoverable), which caps it below High — but it breaks the intended resting-depth guarantee of a core trading surface through a clear off-by-counter bug.

Economic Impact: PoC — a single level is blocked for **~0.5 USDC** of locked capital after the attacker recovers ~127.5 of the ~128 USDC deployed.

Proof of Concept

`test/audit/CLOBLevelCapStaleSlots.t.sol` — real `CLOB` + `Ledger` + `OutcomeToken`.

```
// attacker places 256 min buys at price 5000, then cancels 255 (keeps the front)
for (i in 0..256) ids[i] = clob.placeLimitBuy(5000, 1e6);
for (i in 1..256) clob.cancelOrder(ids[i]);

(uint256 bestBid, uint256 bidSize,,) = clob.getTopOfBook();
assertEq(bestBid, 5000);
assertEq(bidSize, 1e6); // only 1 order active at the level
assertEq(ledger.totalReservedBuy(), 0.5e6); // ~127.5 USDC refunded

vm.prank(victim);
vm.expectRevert(IOrderBook.OrderBookLevelFull.selector);
clob.placeLimitBuy(5000, 1e6); // level "full" with 1 active order

clob.compactBuyQueue(5000, 256); // cannot advance past live front
vm.prank(victim);
vm.expectRevert(IOrderBook.OrderBookLevelFull.selector);
clob.placeLimitBuy(5000, 1e6); // still blocked
```

Run output

```
$ forge test --match-contract CLOBLevelCapStaleSlotsTest -vv
[PASS] test_exploit_staleSlotsBlockLevelCheaply()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

Recommendation

Gate the placement cap on the maintained active counter, which already excludes cancelled/filled slots — apply symmetrically to the sell side (`activeSellCountByPrice` , `CLOB.sol:301`):

```
- uint256[] storage buyQueue = buyOrderIdsByPrice[priceBps_];
- if (buyQueue.length - buyQueueHeadByPrice[priceBps_] >= MAX_RESTING_ORDERS_PER_LEVEL) {
+ if (activeBuyCountByPrice[priceBps_] >= MAX_RESTING_ORDERS_PER_LEVEL) {
    revert IOrderBook.OrderBookLevelFull();
}
```

The matching/finalization scans still walk `length - head` , so keep the compaction discipline that bounds those scans — but the *placement cap* must reflect real resting depth (`activeBuyCountByPrice`), not the raw FIFO window. Ship `test/audit/CLOBLevelCapStaleSlots.t.sol` as the regression guard.

Status

Fixed — The level cap now uses the live active-order count. The correction and queue regressions were merged in [PR #89](#).

[M-08] Two-wallet collusive CLOB buy routes promo spend into a controlled seller's withdrawable winnings

Field	Value
Severity	Medium
Source finding	M-17
GitHub issue	#66
Affected components	<code>CLOB._matchIncomingBuy</code> (no maker == taker / same-controller check on the buy path) · <code>Ledger.debitTakerBuyFill</code> · <code>Ledger.consumeEscrowedSellBasis</code> / <code>_routeSaleProceeds</code> · <code>LedgerBasisLib.routeSalePayout</code>
Confidence	High — confirmed with a passing Foundry PoC on <code>dev-found-bugs</code> (<code>test/audit/PromoCollusiveTakerBuyWash.t.sol</code>)
Attack class	MEVEconomic / ShareAccounting (promo → withdrawable, provenance dropped across a fill hop)

Description

What's wrong

Promo credit is non-withdrawable by design. A user controlling two funded wallets defeats that with an ordinary CLOB fill: a promo-only buyer takes a controlled seller's resting sell priced above the seller's cost basis. The buyer's promo is converted into real pool backing, and the seller's above-basis proceeds are booked as withdrawable winnings — with no external counterparty and no real at-risk outcome. Aggregate withdrawable balance across the two wallets rises by the above-basis overpayment, which the attacker sets by choosing the sell price. The promo that should never be cashable becomes cash in a second wallet.

How (mechanism)

On a taker buy, `_matchIncomingBuy` settles each crossed maker sell with two independent Ledger calls and never checks whether the maker and taker are the same controller (the only guard is `makerOrder.trader == market` for seed asks, a different case):

```
// CLOB.sol:874-898
ILedger(ledger).debitTakerBuyFill(taker, market, outcomeTokenId, notional, takerFee); // buyer spends
_safeTransferOutcome(taker, matchAmount); // buyer gets tokens
if (makerOrder.trader == market) { /* seed ask */ }
else {
  ILedger(ledger).consumeEscrowedSellBasis(
    makerOrder.trader, _orderHash(makerOrderId), matchAmount, 0, notional, 0 // seller realizes proceeds
  );
}
```

- **Buyer leg** — `debitTakerBuyFill` splits the spend deposit-first (`Ledger.sol:418`), so a wallet holding only promo pays entirely from promo (`cp = total`). The notional becomes the buyer's `positionBasis.fromPromo` and `_raiseHoldings(market, total)` (`Ledger.sol:446`) turns that promo into real `marketHoldings` backing.
- **Seller leg** — `consumeEscrowedSellBasis` realizes proceeds against the **seller's own** escrowed basis only (`rd/rw/rp`) and calls `_routeSaleProceeds(user, market, rd, rw, rp, proceeds)` (`Ledger.sol:1040`). The buyer's promo provenance is nowhere in scope. When `proceeds ≥ sellerBasis`, `LedgerBasisLib.routeSalePayout` books the excess to the seller's **winningsBalance** (withdrawable) and `_lowerHoldings(market, proceeds)` pays it out of the `marketHoldings` the buyer's promo just funded.

`marketHoldings` nets flat (`+total` on the buy, `-proceeds` on the sell), but the two wallets' redeemable claims do not: wallet B's winnings rise by `proceeds - B_basis`, funded by wallet A's promo. A is left holding B's former tokens on promo basis — abandoned for free on a losing outcome, or backed by the promo-funded holdings on a winning one. Either way, non-redeemable promo has crossed into a withdrawable bucket with no real counterparty.

Exploit path

Same controller, two funded wallets A (buyer) and B (seller), single outcome, sell price `p` above B's basis `b/token`:

1. B holds `q` outcome tokens at basis `b` and places a resting limit sell at price `p > b` (escrows B's basis via `escrowSellBasis`).
2. A is granted promo and spends down any deposit/winnings so only promo remains.
3. A market-buys (or crosses a limit buy for) `q` against B's sell:
 - `debitTakerBuyFill(A, ..., notional = p·q, ...)` → A's `positionBasis.fromPromo += p·q`, `marketHoldings += p·q` (promo → real backing). A receives the `q` tokens.
 - `consumeEscrowedSellBasis(B, ..., notional = p·q, ...)` → B's basis `b·q` returns to B's redeemable buckets, `winningsBalance[B] += (p - b)·q`, `marketHoldings -= p·q`.
4. B `redeem s` → withdraws real USDC; the `(p - b)·q` is laundered promo. A abandons or holds the tokens as the outcome dictates.

Redeemable created across A+B: `(p - b)·q` withdrawable, sourced entirely from A's promo grant, with no external counterparty.

Impact

- **Theft of operator promo budget** → **withdrawable USDC** across two controlled wallets, no self-match, no external counterparty. The extracted amount is the above-basis overpayment `(p - b)·q`, attacker-chosen via `p`.
- **Under-backed residue / solvency seed**: A's promo-funded position claims `marketHoldings` backing that was net-zeroed by B's payout; on a winning resolution for A's tokens the market owes payouts it cannot cover (`InsufficientMarketHoldings`), shifting the shortfall to honest winners — the same escalation as M-03, bounded by the promo extracted.

- **M-03's shipped fix does not cover this.** A `maker == taker` guard on `_matchIncomingSell` (M-03's recommendation) leaves `_matchIncomingBuy` with two distinct wallets fully open. This finding is the proof that the self-match guard is insufficient and the family needs the provenance-level fix.

Economic Impact: $\sim (p - b) \cdot q$ withdrawable per cycle, bounded by A's promo grant and `maxSubsidyPerWallet` (~\$40–45/wallet on the family's `dev` numbers), repeatable across sybil wallet pairs. >\$10 and >0.01% of the affected budget per iteration → clears the Medium relevance threshold.

Severity — real-world impact × likelihood

- **Impact: Medium.** The directly-extracted asset is the operator's promo budget (marketing capital), per-wallet capped by `maxSubsidyPerWallet`; the under-backed residue can short honest winners but is bounded by the promo extracted — Medium, not clean High.
- **Likelihood: Medium.** `grantPromo` is `onlyRole(OPERATOR_ROLE)` — the attacker cannot self-grant; scaling needs a sybil against the operator's promo distribution and a controlled seller resting above basis (thin ask side / appreciated inventory). Trivial once those hold.
- **Medium × Medium → Medium.** Consistent with the rest of the promo family ([M-01]–[M-05]).

Proof of Concept

`test/audit/PromoCollusiveTakerBuyWash.t.sol` — real `CLOB` + `Ledger` + `OutcomeToken` (the test contract stands in for the Market lifecycle surface, `graduated() == true`). Two wallets: promo-only buyer A, deposit-funded seller B resting above basis. Promo = 50 USDC (`maxSubsidyPerWallet`).

```
// Seller B: deposit-funded $0.10/token position, holds 50 tokens
outcomeToken.mint(seller, ID, 50e6);
ledger.deposit(50e6); // as B
ledger.debit(seller, market, ID, Basis(50e6, 0, 0)); // B basis, deposit consumed

// Buyer A: funded from promo only
ledger.seedPromoBudget(50e6);
ledger.grantPromo(buyer, 50e6, MANUAL);

// B rests a sell at $0.90 (above its $0.10 basis)
vm.startPrank(seller);
outcomeToken.setOperator(address(clob), true);
clob.placeLimitSell(9000, 50e6);
vm.stopPrank();

// Aggregate redeemable across A+B is 0 before the wash.
// A takes B's sell with promo-funded spend
vm.prank(buyer);
clob.marketBuy(50e6, type(uint256).max);

assertEq(ledger.winningsBalance(buyer), 0); // A gains no redeemable
assertGt(ledger.winningsBalance(seller), 39e6); // ~40 USDC promo -> B redeemable winnings
vm.prank(seller);
ledger.redeem(ledger.winningsBalance(seller) + ledger.depositedBalance(seller)); // B withdraws real USDC
// A left holding 50 promo-basis tokens; marketHoldings net-zeroed vs A's claim
```

Run output

```
$ forge test --match-contract PromoCollusiveTakerBuyWashTest -vv
[PASS] test_exploit_collusiveBuyLaundersPromoToSellerWinnings()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The test asserts aggregate redeemable across A+B rises from 0 to ~45 USDC (of which ~40 is laundered promo), A's `winningsBalance` stays 0 while B withdraws real USDC, and A's promo-funded position is left under-backed. After a correct fix (option 1 or 2 above) B's `winningsBalance` must not rise from A's promo spend — the promo fraction routes to

B's `promoBalance` / is bounded by $\sum \text{promo-in} \geq \sum \text{promo-redeemable}$.

Recommendation

Do **not** rely on a `maker == taker` guard — it is bypassed by two wallets. Carry promo provenance across the fill so a promo-funded taker buy cannot mint another wallet's withdrawable winnings. Two coherent options, in preference order:

1. **Class-level fix (preferred, closes the whole family incl. [M-01]–[M-05]):** enforce the system-wide identity $\sum \text{promo-in} \geq \sum \text{promo-redeemable}$. Track promo-funded supply per token id and cap redeemable winnings so promo principal can never cross to `winningsBalance` anywhere — including on the seller leg of a promo-funded taker buy. Ship it as a fuzz invariant.
2. **Local fix (this vector):** thread the taker's promo fraction into `consumeEscrowedSellBasis`, and in `_routeSaleProceeds` / `routeSalePayout` tag the seller's proceeds up to the buyer's promo fraction as non-redeemable (route that slice to the seller's `promoBalance` rather than `winningsBalance`). This preserves promo non-withdrawability across the buyer → seller hop without a self-match assumption.

Ship a two-wallet regression (see PoC) proving a promo-funded taker buy against a controlled above-basis seller cannot raise aggregate redeemable balances except through a real at-risk winning outcome.

Status

Acknowledged / Accepted Risk — Two-wallet CLOB trading remains accepted behavior rather than introducing same-controller restrictions. The finding is formally acknowledged in [issue #66](#).

[M-09] Promo budget laundered into redeemable winnings by buying both outcomes and redeeming the winner

Field	Value
Severity	Medium
Source finding	M-18
GitHub issue	#67
Affected components	<code>Ledger.creditWinnings</code> (root cause, shared with the family) · <code>Ledger.debitPhase1Buy</code> · <code>LedgerBasisLib.routeSalePayout</code> · <code>Market.redeem</code> / <code>Market.calculatePayout</code>
Confidence	High — confirmed with a passing Foundry PoC on the <code>dev</code> branch
Attack class	ShareAccounting / DeltaAccounting (basis-provenance reclassification)

Description

What's wrong

Promo credit is operator-funded marketing money, non-redeemable by design — only a genuine market win is supposed to turn it into withdrawable cash. A user defeats that with no special inventory and no burn: in Phase 1 they buy *both* outcomes of a market with promo, hold to resolution, and redeem the winner. The winner leg's promo returns to `promoBalance` (correct), but the payout is a pari-mutuel share of the whole pool — including the collateral the user sank into the *losing* leg — and that excess over the winner's own basis is booked as withdrawable winnings. The loser-leg promo the user spent becomes the winner-leg profit they cash out.

How (mechanism)

The payout that funds `creditWinnings` is inflated by the loser leg, while the routing only ever sends the winner's own basis back to promo:

- **Both buys record promo basis.** `Market._processDeposit` → `Ledger.debitPhase1Buy` (`Ledger.sol:354`) accepts `spend.fromPromo` and writes it to `positionBasis[user][market][outcomeId].fromPromo` (`Ledger.sol:386-389`), raising `marketHoldings` by the full spend (`Ledger.sol:392`). Buying outcome A and outcome B separately records promo basis on both legs; both legs' collateral lands in `totalPool` (`Market.sol:307-310`).
- **Payout is pari-mutuel — the loser collateral is in the denominator source.** `Market.calculatePayout` returns `userWinningTokens * payoutPool / totalWinningTokens` (`Market.sol:972`), where `payoutPool` is `totalPool` net of resolution rake and therefore includes the loser leg's collateral.
- **Redeem realizes only the winner leg.** `Market.redeem` → `creditWinnings(user, market, correctOutcome, payout)` (`Market.sol:418-421`) clears only the winning outcome's basis (`Ledger.sol:529-541`) and routes `payout` through `routeSalePayout(0,0,a,payout)`. With `payout ≥ a`, the winner promo basis `a` returns to `promoBalance` and the excess `payout - a` books to withdrawable `winningsBalance` (`LedgerBasisLib.sol:115-119`).
- **The loser leg is never realized.** `positionBasis[user][market][losingOutcome]` keeps its `fromPromo` forever (the losing tokens are worthless, so the user never redeems them). That promo is permanently removed from `promoBalance` while an equal amount of its collateral has flowed out to the winner leg as cash.

Net across the round-trip: `promoBalance` falls by the loser-leg amount, `winningsBalance` rises by the same, and the user withdraws real USDC. No genuine profit occurred — the “profit” is the user's own promo, redistributed from the losing leg to the winning leg by the pari-mutuel pool and reclassified as redeemable.

Exploit path (two-outcome market)

Operator seeded the promo budget; attacker holds `P = 50 USDC` promo (the per-wallet cap) and controls / dominates a thin market (anyone can create and self-fund one).

1. **Buy the winner leg** with `25 USDC` promo → `positionBasis[WIN].fromPromo = 25`, `marketHoldings += 25`, `pools[WIN] += 25`.
2. **Buy the loser leg** with `25 USDC` promo → `positionBasis[LOSE].fromPromo = 25`, `marketHoldings += 25`, `pools[LOSE] += 25`. `promoBalance` now `0`.
3. **Market resolves to WIN.** Attacker holds ~all winner tokens, so `calculatePayout ≈ payoutPool ≈ 50` (whole pool, rake aside).
4. **Redeem.** `creditWinnings(attacker, market, WIN, 50): routeSalePayout(0,0,25,50)` → `toPromo = 25` (back to promo), `toWinnings = 25`.
5. **Withdraw** `25` real USDC — the operator's promo seed. Loser-leg `25` promo basis is stranded, permanently gone from `promoBalance`.

Repeat with a fresh wallet to launder another ~\$25.

Realistic-conditions note. Clean extraction requires the attacker to hold a dominant share of the *winning* side. In a liquid two-sided market their loser-leg collateral funds honest winners too, diluting or reversing the take — so the exploit is bounded to thin / self-funded pools. This is the likelihood cap that puts the finding at Medium.

Impact

- **Operator / protocol:** Reclassification of the seeded promo budget into withdrawable USDC — ~\$1 cash per \$1 of loser-leg promo, capped at ~\$25/wallet but unbounded across wallets.
- **Funding-model break:** promo is supposed to convert to redeemable value only through a resolved win on a genuinely-held position; this converts the *losing* leg's promo with no market exposure on that leg.
- **§2.4 solvency stays green** (`totalPromo` falls as `totalWinnings` rises), so the solvency harness does not catch it — the loss is to the promo budget, not honest winners (the payout is drawn from `marketHoldings` the attacker's own buys funded).

Economic Impact: PoC launders **25 of 50 USDC** promo into redeemable winnings per wallet, withdrawn as real USDC.

Severity — real-world impact × likelihood

- **Impact: Medium.** Asset at risk is the operator's **promo budget** (marketing capital), not user principal or protocol solvency. Per-wallet take is bounded by `maxSubsidyPerWallet` (50 USDC), and the winner payout is paid from `marketHoldings` the attacker's own promo funded — no honest-winner shortfall.
- **Likelihood: Medium.** `grantPromo` is `onlyRole(OPERATOR_ROLE)` — the attacker cannot self-grant. Clean extraction additionally needs an **attacker-dominated winning side** (self-funded thin market) so the loser-leg collateral flows back to the attacker rather than honest winners. Trivial mechanics once promo is granted, but the gating + per-wallet cap + pool-domination requirement bound reachability.
- **Medium × Medium → Medium.** The submitter's "≤2 tx, no admin keys → High" reads the mechanics, not the blast radius: bounded promo budget, operator-fed, requires a dominated pool. Likelihood is not ignored here the way impact-only Sherlock scoring would.

Proof of Concept

`test/audit/PromoBuyBothSidesHoldRedeem.t.sol` (real `Ledger` + `YieldVault`; test contract is the registered market and supplies `payout` exactly as `Market.redeem` → `creditWinnings` does after a self-dominated pool resolves). The pari-mutuel payout inflation itself is established by reading `Market.calculatePayout` (`Market.sol:972`); the PoC isolates the Ledger routing it feeds.

```
function test_control_singleLeg_promoStaysLocked() public {
    _seedAndGrantPromo(attacker, LEG);
    _phase1BuyWithPromo(attacker, WIN, LEG);
    ledger.creditWinnings(attacker, market, WIN, LEG); // payout == own basis
    assertEq(ledger.promoBalance(attacker), LEG);      // promo round-trips back
    assertEq(ledger.winningsBalance(attacker), 0);     // nothing redeemable
}

function test_exploit_buyBothSides_laundersLoserPromoToWinnings() public {
    _seedAndGrantPromo(attacker, PROMO);
    _phase1BuyWithPromo(attacker, WIN, LEG);           // winner-leg promo basis
    _phase1BuyWithPromo(attacker, LOSE, LEG);          // loser-leg promo basis
    uint256 payout = LEG + LEG;                       // whole pool, rake modeled 0
    ledger.creditWinnings(attacker, market, WIN, payout);
    assertEq(ledger.promoBalance(attacker), LEG);      // winner basis back to promo
    assertEq(ledger.winningsBalance(attacker), LEG);   // loser promo -> redeemable
    (, uint128 loserPromo) = ledger.positionBasis(attacker, market, LOSE);
    assertEq(loserPromo, uint128(LEG));                // loser basis stranded
    vm.prank(attacker); ledger.redeem(LEG);            // real USDC out
}
```

Run output

```
$ forge test --match-contract PromoBuyBothSidesHoldRedeemTest -vv
[PASS] test_control_singleLeg_promoStaysLocked()
[PASS] test_exploit_buyBothSides_laundersLoserPromoToWinnings()
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

The control proves intended behavior (single-leg promo win round-trips back to `promoBalance`); the exploit proves the loser leg's promo becomes withdrawable.

Recommendation

The cluster fix from [M-01] / [M-02] closes this path too — enforce that promo can never leave as more than it entered:

1. **Cluster invariant** — maintain $\sum \text{promo-in} \geq \sum \text{promo-redeemable}$ per `(user, market)` (a fuzz/echidna invariant plus a runtime guard). The loser-leg promo cannot be reclassified as winnings if the winner-leg payout

routing is capped at the user's net promo contribution.

2. **Local mitigation** — when `creditWinnings` routes a payout whose basis contains promo, cap the `winningsBalance` credit at genuine profit relative to the user's *total* promo basis across all outcomes of the market, not just the winning leg. Route the loser-leg-funded excess back to `promoBalance` instead of winnings.

Either fix must keep the control test passing (single-leg promo win returns promo to `promoBalance`, profit above true basis still redeemable) while making the buy-both-sides exploit credit the loser-leg amount back to `promoBalance`. Ship `test/audit/PromoBuyBothSidesHoldRedeem.t.sol` as the regression guard.

Status

Acknowledged / Accepted Risk — The team accepts this promo behavior as part of the chosen promo-volume model. The finding is formally acknowledged in [issue #67](#).

Informational Findings

[I-01] Market-fund bridges are asymmetric on yield principal — `convertDepositToMarketHoldings` pays for a no-op yield sync every call

Field	Value
Severity	Informational
Source finding	I-05
GitHub issue	#69
Affected components	<code>Ledger.convertDepositToMarketHoldings</code> · <code>Ledger.convertMarketHoldingsToDeposit</code> · <code>Ledger._decreaseYieldPrincipal</code> / <code>_settleYield</code> / <code>_syncYield</code>
Confidence	High — verified against every <code>yieldPrincipal</code> credit path on <code>dev</code>
Attack class	— (accounting hygiene / gas)

Description

What's wrong

The two functions that bridge a market's funds between `marketHoldings` and `depositedBalance` are asymmetric with respect to yield principal. The holdings → deposit direction never credits yield principal; the deposit → holdings direction asks to decrease it. Because a market never accrues any yield principal in the first place, the decrease is always a no-op — but it still triggers a full `_settleYield` → `_syncYield` (an external call into the YieldVault) on every call.

How (mechanism)

```
// holdings -> deposit: no _increaseYieldPrincipal
function convertMarketHoldingsToDeposit(uint256 amount) external onlyRegisteredMarket ... {
    _lowerHoldings(msg.sender, amount);
    _creditUser(msg.sender, amount, 0, 0); // <-- no yield-principal credit
}

// deposit -> holdings: _debitUserAmounts(..., true) -> _decreaseYieldPrincipal
function convertDepositToMarketHoldings(uint256 amount) external onlyRegisteredMarket ... {
    _debitUserAmounts(msg.sender, amount, 0, 0, true); // <-- true => decrease yield principal
    _raiseHoldings(msg.sender, amount);
}
```

`_decreaseYieldPrincipal` runs the settle/sync **before** the clamp, so a zero-principal market still pays for it:

```
function _decreaseYieldPrincipal(address user, uint256 amount) internal {
    if (amount == 0) return;
    _settleYield(user); // -> _syncYield() -> yieldVault.syncYield() (EXTERNAL CALL)
    uint256 principal = yieldPrincipal[user]; // market principal == 0
    if (amount > principal) amount = principal; // amount -> 0
}
```

```

    if (amount == 0) return; // returns; the decrease was a no-op
    ...
}

```

The caller is always a market (`onlyRegisteredMarket`), so the decrease is *unconditionally* a no-op, yet every call incurs the external `yieldVault.syncYield()`.

Why a market's `yieldPrincipal` is provably always zero

`_increaseYieldPrincipal` has exactly four call sites; none fires with `user == market`:

- `deposit(msg.sender)` (`Ledger.sol:202`) — markets fund via `depositForMarket` / `convertMarketHoldingsToDeposit`, never `deposit`.
- `creditBurnProceeds(user)` (`:641`) — `burnCompleteSet` is a user path.
- `distributeFillFee(maker)` (`:764`) — guarded by `if (maker == market) revert InvalidFeeRecipient()` (`:722-723`); a market maker never reaches the increase.
- `_routeSaleProceeds(user)` (`:1245`) — reachable only if a market realizes a position. A market's only position source is seeded-bid fills, and those **burn** the tokens (`makerOrder.trader == market → OutcomeToken.burn`), so the market holds nothing to redeem and never reaches `creditWinings` / `creditSaleProceeds` with `user == market`.

On the debit side, `reserveBuyCredit` passes `_debitUser(user, spend, false)` (`:814`), so even a market's seed-bid reservation never touches principal. Hence `yieldPrincipal[market] == 0` at all times, and the `true` in `convertDepositToMarketHoldings` is a guaranteed no-op decrease. The incidental `_syncYield` is harmless (it only attributes surplus slightly earlier), so there is **no correctness bug today** — only wasted gas and a misleading asymmetry.

Impact

- **Wasted gas:** every `convertDepositToMarketHoldings` pays for a cross-contract `yieldVault.syncYield()` that does nothing for the market's (zero) principal.
- **Code-clarity / latent asymmetry:** the two bridges disagree on principal handling. It reads as if markets participate in yield accrual when they do not, and it hides a dormant leak: *if* a market ever did hold principal, the holdings $\leftrightarrow$ deposit round-trip would silently lose it (only the decrease direction touches principal).

No fund loss, no exploit.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Given today's "markets carry no yield principal" invariant, pass `false` — it is strictly correct and matches the converse (`convertMarketHoldingsToDeposit` does not touch principal either):

```

- _debitUserAmounts(msg.sender, amount, 0, 0, true);
+ _debitUserAmounts(msg.sender, amount, 0, 0, false); // markets carry no yield principal

```

Caveat — decide the [H-01] fix first (they share one question)

This is coupled to how [H-01] (yield earned on market-owned float) is remediated — both turn on “*should market-owned float participate in yield accrual?*”:

- If H-01 is fixed by excluding market float from the yield surplus (markets stay principal-less) $\rightarrow$ `false` is permanently correct; take the gas win.

- If H-01 is fixed by attributing yield principal to market float → markets *would* accrue principal, and both bridges then need symmetric handling: `convertMarketHoldingsToDeposit` must *add* `_increaseYieldPrincipal`, and `convertDepositToMarketHoldings` must keep the *real* decrease (`true`). In that world, flipping to `false` becomes a **latent leak**.

So do not flip in isolation: if H-01 is closed by excluding float, set `false`; if by attributing float yield to markets, keep `true` and add the missing `_increaseYieldPrincipal` to the converse instead.

Status

Fixed — Market holdings now carry yield principal symmetrically across both bridge directions. The correction was merged in [PR #88](#).

[I-02] `MarketLens.calculatePotentialPayout` previews the pre-fold `totalPool`, under-reporting the resolved payout by `pendingSeedPoolInflow`

Field	Value
Severity	Informational
Source finding	I-06
GitHub issue	#70
Affected components	<code>MarketLens.calculatePotentialPayout</code> · <code>Market._completeResolution</code> / <code>_foldPendingSeedIntoTotalPool</code>
Confidence	High — exploit-free; verified end-to-end against the resolution fold and the settlement path
Attack class	— (read-only correctness / display parity)

Description

What's wrong

The frontend payout preview reads the market's pool *before* a graduation-time balance is folded in, so it quotes a smaller potential payout than winners actually receive at resolution. The gap is one-directional — the preview always under-states, never over-states — so no one relying on it is economically harmed; they simply receive more than shown. This is a display inconsistency against the function's own stated intent, not a settlement bug.

How (mechanism)

`calculatePotentialPayout` derives its pool from the live cached total and never adds `pendingSeedPoolInflow`:

```
// MarketLens.sol:113-118
uint256 totalPoolValue = marketState.totalPool();           // pre-fold value
if (totalPoolValue == 0) return 0;
uint256 rake = (totalPoolValue * cache.rakePercentage) / MarketConstants.BPS_DENOMINATOR;
uint256 hypotheticalPayoutPool = totalPoolValue - rake;      // <-- omits pendingSeedPoolInflow
```

At resolution, `_completeResolution` folds that balance in **first**, then computes the pool the payout is actually drawn from:

```
// Market.sol:1299 -> 1239
_foldPendingSeedIntoTotalPool(); // totalPool += pendingSeedPoolInflow; pendingSeedPoolInflow = 0
...
// Market.sol:1318-1328
uint256 totalPoolValue = totalPool; // post-fold value
payoutPool = nowWinners ? 0 : totalPoolValue - rake;
```

The settled claim in `Market.calculatePayout` reads that post-fold `payoutPool` (`Market.sol:972`), independent of the Lens. So while `pendingSeedPoolInflow > 0`, the preview's pool is smaller than the settlement pool by exactly that balance.

`pendingSeedPoolInflow` is non-zero across the whole Phase-2 window of any graduated market. It is seeded at graduation with the retained seed dust + askFund (`seedFundValue - totalBidNotionalSent`, `Market.sol:1658`) and grows on every seeded-bid refund via `onSeededBidRefund` (`Market.sol:697`). So this is the common case for graduated markets, not an edge case.

The Lens already matches the resolution *denominator* — it subtracts Market-held tokens from `totalSupply` (`MarketLens.sol:128-130`) with a comment stating the goal is consistency “with the actual resolution denomination.” It matched the supply side but left the *pool* (numerator) on the pre-fold value, so the two halves disagree.

Impact

- **External consumers / frontend:** the previewed payout for a winning outcome is smaller than the settled payout by a factor of `pendingSeedPoolInflow / totalPool` — the retained seed dust + askFund plus any refunded seed bids, persistently non-zero from graduation until resolution.
- **No fund-handling consumer.** `MarketLens` is a standalone read-only helper; no contract under `src/` calls `calculatePotentialPayout`. Settlement (`Market.calculatePayout` → `payoutPool`) never touches the Lens, so the wrong preview cannot reach the payout math.
- **Conservative direction.** The fold only *adds* to the pool, so `payoutPool_actual = (totalPool + P) · (1 - rake) ≥ totalPool · (1 - rake) = preview`. The preview is a lower bound; users are never misled into over-committing.

No fund loss, no over-reporting path, no exploit.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Add the pending inflow to the preview pool so the Lens mirrors `_completeResolution`:

```
uint256 totalPoolValue = marketState.totalPool();
+ // Mirror `_completeResolution`, which folds `pendingSeedPoolInflow` into
+ // `totalPool` before computing `payoutPool`. Omitting it makes the preview
+ // under-report the resolved payout for the whole Phase-2 window.
+ totalPoolValue += marketState.pendingSeedPoolInflow();
if (totalPoolValue == 0) return 0;

uint256 rake = (totalPoolValue * cache.rakePercentage)
/ MarketConstants.BPS_DENOMINATOR;
uint256 hypotheticalPayoutPool = totalPoolValue - rake;
```

`pendingSeedPoolInflow()` is a public state variable on `Market` (`Market.sol:123`). Confirm the `IMarket` interface `MarketLens` imports declares the getter alongside `totalPool()`; add it if missing.

Scope note: this is a cosmetic preview fix and does not affect settlement or any on-chain balance.

Status

Fixed — The payout preview includes pending seed inflow and claimable market yield. The correction was merged in [PR #85](#).

[I-03] A paused Ledger stalls graduated-market finalization — order unwinds are pause-exempt but the completion step is pause-gated

Field	Value
Severity	Informational
Source finding	I-07
GitHub issue	#71
Affected components	Market.processFinalizationBatch / _completeResolution / _reclaimSeedDepositToHoldings / _payTreasury · Ledger.convertDepositToMarketHoldings / creditTreasury (vs. pause-exempt Ledger.releaseReservedBuy / releaseEscrowedSellBasis)
Confidence	High — every pause modifier and the full finalization call path verified end-to-end
Attack class	— (liveness / pause-gating asymmetry; admin-trust)

Description

What's wrong

A graduated market's finalization is a two-part process: first the CLOB order books are drained (unfilled seed bids refunded), then a final accounting step routes the reconciled pool to holdings/treasury. The order-unwind half is deliberately allowed to run while the Ledger is paused; the accounting half is not. So a graduated market can enter pending-finalization, fully drain its books during a pause, and then be unable to *complete* — every attempt to finish reverts at the pause-gated accounting call. The market sits in pending-finalization, and winners are held, until governance unpauses. Nothing is lost and no state is corrupted; the stall lasts exactly as long as the admin keeps the Ledger paused.

How (mechanism)

For a *graduated* market, `resolve()` / `cancel()` do not finalize inline — they call `_startPendingFinalization` and return (`Market.sol:363-368` , `396-401`). `processFinalizationBatch` (`Market.sol:824`) then drains each book via `cancelOrdersRange` , which fires `onSeededBidRefund` and refunds unfilled seed-bid collateral into `Ledger.depositedBalance[market]` through the **pause-exempt** `releaseReservedBuy` (`Ledger.sol:889` , `no whenNotPaused`) / `releaseEscrowedSellBasis` (`Ledger.sol:1255` , `no whenNotPaused`).

When the last book completes, the same call falls through to `_finalizeResolvedMarket` / `_finalizeCancelledMarket` (`Market.sol:873-877`) → `_completeResolution` , which makes two **pause-gated** Ledger calls:

```
// Market._completeResolution
_reclaimSeedDepositToHoldings(); // -> Ledger.convertDepositToMarketHoldings (whenNotPaused) Market.sol:1304
...
_payTreasury(rake);              // -> Ledger.creditTreasury (whenNotPaused) Market.sol:1338
```

Both `convertDepositToMarketHoldings` (`Ledger.sol:1334`) and `creditTreasury` (`Ledger.sol:695`) carry `whenNotPaused` . Under a pause:

- `_reclaimSeedDepositToHoldings` reverts whenever a seed residual exists (`depositedBalance[market] > 0` from the refunds just booked).
- `_payTreasury` → `creditTreasury` reverts on any non-zero rake **even with zero residual**, so the reclaim step is sufficient but not necessary — a graduated resolve with any non-zero pool cannot complete while paused. The cancel-no-refund path (`_payTreasury(accountedBacking)` , `Market.sol:1371`) reverts identically.

The reverting final batch rolls back cleanly: `pendingFinalizationBookIndex` and the per-book cursors (`Market.sol:850/860`) are inside the reverted tx and never advance, so on unpaused `processFinalizationBatch` resumes from the last committed cursor with no double-count of `onSeededBidRefund` . Finalization then completes normally.

Impact

- **Winners / cancelled depositors:** payouts on a graduated market are *delayed* for the duration of a Ledger pause — settlement cannot complete until governance unpauses. Delayed, never denied.
- **No fund loss, no permanent lock, no corruption.** Intermediate batches commit consistent, liability-neutral basis relocations; the completing batch reverts atomically; state resumes coherently after unpaused.
- **Admin-gated trigger.** `pause()` / `unpause()` are `onlyRole(DEFAULT_ADMIN_ROLE)` (`Ledger.sol:1418/1423`). No attacker path; the stall window equals the admin-chosen pause duration, and the same admin ends it. A permanent stall requires admin abandonment — a centralization/QA concern, not a code defect.

This is the intended semantics of a pause: it freezes \$2.4 counter-moving operations (`marketHoldings` / `deposited` / `treasury`), while the pause-exempt releases are counter-neutral. Blocking finalization under pause is what a pause is for — hence Informational, not a vulnerability.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Decide and document the intended pause semantics for graduated finalization; two coherent options:

1. **Confirm finalization is meant to be frozen under pause** (status quo) — add a NatSpec note on `processFinalizationBatch` / `_completeResolution` stating that a Ledger pause halts completion by design, and surface a clear revert (e.g. a `LedgerPaused` guard checked up front in `processFinalizationBatch` when the batch would complete) so operators see an explicit reason rather than an opaque `whenNotPaused` revert deep in the accounting step.
2. **Allow graduated markets to finalize during a pause** (if winners should not be held hostage by a pause) — route the completion accounting through a pause-exempt, still liability-neutral bridge. `convertDepositToMarketHoldings` and `creditTreasury` only move \$2.4 counters within the market's own backing (no USDC leaves the vault), the same neutrality that justifies exempting the releases; a dedicated finalization-only path could carry that exemption without weakening the pause's protection of user-facing deposit/redeem.

Scope note: liveness/design clarification. No balance or solvency change either way.

Status

Acknowledged / Accepted Design — [PR #86](#) adds an explicit pause error but intentionally leaves finalization blocked until the Ledger is unpaused.

[I-04] `MarketLens.calculatePotentialPayout` divides by live outcome supply post-resolution — overquotes remaining winners as others redeem

Field	Value
Severity	Informational
Source finding	I-09
GitHub issue	#73
Affected components	<code>MarketLens.calculatePotentialPayout (MarketLens.sol:128-132)</code> · vs. <code>settlement Market.calculatePayout (Market.sol:969-972) / Market.redeem (Market.sol:406-421)</code>
Confidence	High — both denominators and the redeem burn path traced end-to-end
Attack class	— (read-only correctness / preview ↔ settlement parity)

Description

What's wrong

After a market resolves, real winner payouts are computed against a denominator frozen at resolution, so every winner's claim is fixed the moment the market resolves. The payout preview does not freeze — it recomputes the denominator from live outcome-token supply on every call. As winners redeem and their winning tokens burn, the live supply shrinks while the preview's pool stays put, so the preview inflates the potential payout for the winners who haven't redeemed yet. It is a display-only skew: the actual claim each remaining winner receives is unchanged, because settlement ignores the live supply entirely.

How (mechanism)

Settlement uses a fixed snapshot. `Market.calculatePayout` divides by `winningTokenSupplyAtResolution ( Market.sol:969-972 )`, captured once at resolution (`Market.sol:1289 = _externalOutcomeSupply(correctOutcome_)`). `redeem()` computes `payout = calculatePayout(msg.sender)`, burns the winner's tokens via `_burnOutcomeTokensOnRedeem ( Market.sol:414 )`, and draws `payout` from `marketHoldings` — it never mutates the snapshot or `totalPool ( Market.sol:406-421 )`.

The Lens recomputes live. `calculatePotentialPayout` has no `resolved` branch; it divides by live external supply and sources its numerator from `totalPool()`, which redemptions do not decrement:

```
// MarketLens.sol:128-132
uint256 totalOutcomeTokens = OutcomeToken(cache.outcomeToken)
    .totalSupply(outcomeIndex_)
- OutcomeToken(cache.outcomeToken).balanceOf(market_, outcomeIndex_); // live, shrinks on redeem
if (totalOutcomeTokens == 0) return 0;
return (userOutcomeTokens * hypotheticalPayoutPool) / totalOutcomeTokens;
```

Worked example (matches the report):

- `payoutPool = 900`, `winningTokenSupplyAtResolution = 1000`, Alice = 500, Bob = 500.
- Real payout each: $500 \times 900 / 1000 = 450$ (fixed by the snapshot).
- Alice redeems first; her 500 tokens burn → live external supply falls to 500.
- Bob's **real** payout is still 450 (snapshot denominator 1000).

- Bob's **Lens** preview becomes $500 \times 900 / 500 = 900$ — a 2× overquote. The skew widens as more winners redeem.

(The numerator `hypotheticalPayoutPool` is `totalPool - rake`, which equals `payoutPool` here since the example has no `pendingSeedPoolInflow`; the residual numerator gap is the separate I-02 issue.)

Impact

- **Remaining (un-redeemed) winners:** post-resolution, `calculatePotentialPayout` reports an inflated claimable that grows as other winners redeem — up to arbitrarily large as live supply approaches the caller's own balance.
- **No fund loss, no on-chain consumer.** `calculatePotentialPayout` is a read-only helper; nothing under `src/` calls it (see I-02). Redemption runs entirely through `Market.calculatePayout` → `payoutPool` → `marketHoldings`, all keyed to the fixed snapshot. A winner cannot realize the inflated figure — `redeem()` recomputes the correct value regardless of what the preview showed.
- **Direction note:** unlike I-02 (which under-quotes conservatively across all phases), this over-quotes and only after resolution — the more misleading direction for a user reading the preview, which is why it is worth fixing even though it is display-only.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

When the market is `resolved`, mirror settlement instead of recomputing from live supply. Simplest and least drift-prone is to delegate to the authoritative post-finalization value:

```
// MarketLens.calculatePotentialPayout, early in the body
if (IMarket(market_).resolved()) {
    return IMarket(market_).calculatePayout(user_); // snapshot denominator + payoutPool
}
if (IMarket(market_).cancelled()) {
    return IMarket(market_).calculatePayout(user_); // cancelled-refund branch
}
// ...existing pre-resolution preview...
```

If a self-contained Lens computation is preferred over delegation, use `winningTokenSupplyAtResolution` as the denominator (and `payoutPool` as the numerator, for the winning outcome only) once resolved. Either collapses the preview onto the value the user will actually claim and removes the live-supply skew.

Scope note: this is fix-distinct from I-02 (numerator vs. denominator; different code sites), so per the bundling rule it submits separately even though both live in `calculatePotentialPayout`. No solvency or balance change.

Status

Fixed — Post-resolution previews delegate to the frozen settlement snapshot. The correction was merged in [PR #85](#).

[I-05] `getMarketStatuses` / `getMarketDataStatus` report the stored `closed` flag, ignoring time-based closure — a market untradeable by time still reads `closed = false`

Field	Value
Severity	Informational
Source finding	I-10
GitHub issue	#74
Affected components	<code>MarketFactory.getMarketStatuses (MarketFactory.sol:421)</code> · <code>MarketLens.getMarketDataStatus (MarketLens.sol:272)</code> · vs. the trading gate <code>Market._onlyOpen (Market.sol:1116-1134)</code>
Confidence	High — both view reads and the trading gate traced end-to-end
Attack class	— (read-only correctness / status ↔ behaviour parity)

Description

What's wrong

A market stops accepting trades the moment its close time passes, but the two status helpers that frontends and indexers read never reflect that. They return the raw stored `closed` boolean, which only flips when the owner explicitly closes, resolves, or cancels the market. So in the window after `closeTime` has elapsed but before any owner action, both helpers report `closed = false` while every trade reverts `MarketNotOpen`. The field is documented as “Whether the market is closed for trading” (`Market.sol:94`) — it under-reports exactly that.

How (mechanism)

The stored flag and the trading gate are two independent notions of “closed”:

- **Stored flag.** `Market.closed` is set `true` only by an explicit action — `closeEarly ( Market.sol:382 )`, a finalization path, or the graduation-close path (`Market.sol:1267-1269`). Nothing sets it from the clock.
- **Trading gate.** `_onlyOpen` (guarding `buyOnPhase1`, `mintCompleteSet`, and the sell/CLOB entrypoints) reverts on time, independent of the flag:

```
// Market.sol:1124-1133
if (timing.closeTime != 0 && block.timestamp >= timing.closeTime) revert MarketNotOpen();
if (timing.closeTime == 0 && timing.resolutionTime != 0
    && block.timestamp >= timing.resolutionTime) revert MarketNotOpen();
```

- **The helpers read only the flag.** `MarketFactory.getMarketStatuses` sets `closed[i] = m.closed()` (`MarketFactory.sol:421`); `MarketLens.getMarketDataStatus` sets `closed_ = marketContract.closed()` (`MarketLens.sol:272`). Neither consults `timing.closeTime` / `timing.resolutionTime`.

So for `closeTime <= block.timestamp < <owner closes>`, the market is untradeable yet both helpers report `closed = false`.

Impact

- **Frontends / integrators:** an integrator that treats `closed == false` as “tradeable” shows the market as open and lets users submit buys/sells that revert on-chain (`MarketNotOpen`) — wasted gas and a confusing UX, no worse.

- **No fund loss, no on-chain consumer.** Both functions are external `view` aggregators; grepping every caller under `src/` finds only tests — nothing state-changing branches on them. There is also no time-aware `isTradeable` / `isOpen` view for integrators to use instead, so the optimistic flag is the only status signal available.
- **No exploit.** The market is genuinely untradeable in this window; the stale view hides nothing exploitable — it is optimistic, not permissive. The trading gate itself is correct.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Expose the effective status rather than the raw flag. Compute it in the view (do not mutate state) by OR-ing the stored flag with the time gate that `_onlyOpen` uses:

```
// in each helper, per market
bool effClosed = m.closed()
  || (closeTime != 0 && block.timestamp >= closeTime)
  || (closeTime == 0 && resolutionTime != 0 && block.timestamp >= resolutionTime);
```

Prefer adding a dedicated `Market.isTradeable()` / `effectiveClosed()` view that mirrors `_onlyOpen`'s time logic in one place, and have both helpers delegate to it — so the status surface cannot drift from the trading gate again. Keep the raw `closed` field available for callers that specifically need the owner-action flag.

Scope note: fix-distinct from I-02/I-04 (different functions, different field), so it submits separately per the bundling rule. No solvency or balance change.

Status

Fixed — Status helpers now account for time-based closure. The correction was merged in [PR #85](#).

[I-06] Cancelled-market refund via `redeem()` is pause-gated while CLOB order unwinds are pause-exempt — refunds are deferred, not blocked, during a Ledger pause

Field	Value
Severity	Informational
Source finding	I-11
GitHub issue	#75
Affected components	<code>Market.redeem</code> → <code>_refundCancelledViaLedger</code> → <code>Ledger.creditSaleProceeds (Ledger.sol:664, whenNotPaused)</code> · vs. the pause-exempt <code>Ledger.releaseReservedBuy (Ledger.sol:889) / releaseEscrowedSellBasis (Ledger.sol:1255)</code>
Confidence	High — every pause modifier and the cancelled-refund call path verified end-to-end
Attack class	— (liveness / pause-gating asymmetry; admin-trust)

Description

What's wrong

The CLOB deliberately lets open-order unwinding settle during a security pause — `releaseReservedBuy` and `releaseEscrowedSellBasis` omit the pause guard so cancels can clear. The terminal user-facing refund does not get the same treatment: a cancelled market's `redeem()` routes every outcome through `creditSaleProceeds`, which keeps `whenNotPaused`. So when an admin pauses the Ledger and cancels a market with `refund_ = true` to protect users, each user's `Market.redeem()` reverts `EnforcedPause` until governance unpauses. The refund is deferred, not lost — but the asymmetry means the “cancel + pause” pairing meant to protect users temporarily blocks the very refund it sets up.

How (mechanism)

For a non-graduated market, `cancel(true)` finalizes inline — `_finalizeCancelledMarket` sets `cancelled = true` and snapshots supply (`Market.sol:1342-1361`) **without touching any pause-gated Ledger function**, so the cancel itself succeeds under a pause. The refund is a per-user pull, deferred to `redeem()`:

```
// Market._refundCancelledViaLedger (called from redeem() when cancelled)
OutcomeToken(outcomeToken).burnFromMarket(user, i, userBalance);
if (share > 0) {
    ILedger(ledger).creditSaleProceeds(user, address(this), i, share); // Market.sol:462-463
}
```

`creditSaleProceeds` carries `whenNotPaused` (`Ledger.sol:664`). Under a Ledger pause every per-user refund reverts, while the CLOB-cancel releases run through (`releaseReservedBuy` / `releaseEscrowedSellBasis` are pause-exempt by design, `Ledger.sol:888` “Runs while paused so cancels can settle”).

Why the real impact is small. The releases and `creditSaleProceeds` are both *internal-bucket* moves — neither sends USDC out. Actual withdrawal is `Ledger.redeem(amount)` (`Ledger.sol:219`), which is *also* `whenNotPaused`. So during the pause **neither cohort can withdraw a cent**: the CLOB-cancel cohort gets its basis tidied back into internal buckets but still can't cash out, and the cancelled-refund cohort simply claims after unpause. The pause freezes value at the withdrawal boundary symmetrically; the asymmetry is only in *when the internal bookkeeping runs*, which no user can monetize mid-pause.

Impact

- **Cancelled-market users:** refunds are *delayed* for the duration of the Ledger pause — `redeem()` reverts until unpause, then settles normally. Delayed, never denied.
- **No fund loss, no permanent lock.** `cancel(true)` finalizes cleanly under pause; the snapshot (`cancelledOutcomeSupplyAtCancellation`) fixes each user's share, so the post-unpause refund is unchanged by the delay.
- **Admin-gated trigger, no attacker path.** `pause()` / `unpause()` are `onlyRole(DEFAULT_ADMIN_ROLE)` (`Ledger.sol`); the stall window equals the admin-chosen pause duration. A permanent block requires admin abandonment (centralization/QA, not a code defect).
- **Escalation condition (worth noting):** if a claim deadline exists after which unclaimed cancelled-refund collateral sweeps elsewhere, a pause spanning most of that window could compress users' effective claim time. Confirm the cancelled-refund path has no such deadline, or that it is paused/extended alongside the Ledger.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Pick one and document it, consistent with the [I-03] pause-semantics decision:

1. **Confirm terminal refunds are meant to freeze under pause** (status quo) — add a NatSpec note on `redeem()` / `_refundCancelledViaLedger` stating a Ledger pause halts cancelled-refund claims by design, and surface an explicit up-front revert reason rather than an opaque `EnforcedPause` deep in `creditSaleProceeds`.
2. **Let cancelled refunds settle during a pause** (if users should not be held) — `creditSaleProceeds` on a *cancelled* market is a liability-neutral relocation of the user's own accounted backing (same neutrality that justifies exempting the releases). Route the cancelled-refund credit through a pause-exempt, still- `whenNotPaused` -for- withdrawal path: the internal credit clears during the pause, and the actual USDC withdrawal (`Ledger.redeem`) still waits for un-pause — so users can queue their refund without weakening the pause's protection of the withdrawal boundary.

Scope note: fix-distinct from I-03 (different function, different call site), so it submits separately per the bundling rule. No balance or solvency change either way.

Status

Acknowledged / Accepted Design — [PR #86](#) adds an explicit paused-refund error while intentionally retaining the pause gate.

[I-07] `cancel()` has no time-gate or delay while `resolve()` enforces `resolutionDelay` — owner can cancel a market at any point, and users have no permissionless safety valve

Field	Value
Severity	Informational
Source finding	I-12
GitHub issue	#76
Affected components	<code>Market.cancel (Market.sol:389-402 , no time guard) · MarketFactory.cancelMarket (MarketFactory.sol:222 , onlyOwner) · VS. Market.resolve (Market.sol:343-362 , enforces resolutionDelay)</code>
Confidence	High — both finalization paths and their modifiers traced end-to-end
Attack class	— (centralization / owner-trust; missing symmetry guard)

Description

What's wrong

Resolution and cancellation are the two terminal paths, but they are gated asymmetrically. `resolve()` cannot fire until a `resolutionDelay` has elapsed since the last qualifying trade — an anti-front-run cooldown. `cancel()` has no equivalent: no check that `closeTime` has passed, no minimum open-time, no delay after the last trade. The owner can cancel any market at any moment. `cancel(false)` sweeps the entire accounted backing to the treasury and pays holders nothing; `cancel(true)` refunds principal but can still terminate a market whose outcome is already decided off-chain before on-chain resolution runs. Users have no permissionless cancel path of their own as a safety valve if the owner stalls.

How (mechanism)

`resolve()` enforces the cooldown (`Market.sol:343-362`):

```
uint256 baselineTs = resolutionActivitySnapshotTs != 0
    ? resolutionActivitySnapshotTs : _getLatestActivityTimestamp();
if (baselineTs > 0 && timing.resolutionDelay > 0) {
    if (block.timestamp < baselineTs + timing.resolutionDelay) revert MustWaitAfterLastTrade();
}
```

`cancel()` has no such gate — it goes straight to finalization (`Market.sol:389-402`):

```
function cancel(bool refund_) external onlyFactory onlyNotFinalizing onlyNotFinalized {
    if (graduated) { _startPendingFinalization(... CANCEL ...); return; }
    _finalizeCancelledMarket(refund_); // no closeTime / resolutionTime / delay check
}
```

`cancelMarket` is `onlyOwner` (`MarketFactory.sol:222`). On the no-refund branch, `_finalizeCancelledMarket(false)` routes the full accounted backing to treasury — `_payTreasury(totalPool + seedFund + pendingSeedPoolInflow)` (`Market.sol:1368-1371`) — and zeroes the pools, so winners receive nothing. There is no user-callable cancel anywhere; every terminal path is `onlyFactory` ← `onlyOwner`.

Impact

- **Owner-trust dependency.** Both harmful readings — sweeping the pool to treasury via `cancel(false)`, or front-running an off-chain-decided outcome via `cancel(true)` before `resolve()` — require the **trusted owner** to act against users. Under the platform trust model (owner assumed honest) these are not attacker-reachable, hence Informational/QA rather than H/M.
- **No user safety valve.** If the owner never resolves and never cancels, users have no permissionless exit for a stuck market. This is the sharper, defensible half of the observation — a liveness/self-custody gap independent of owner malice.
- **Design asymmetry.** The `resolutionDelay` cooldown that protects resolution from last-second manipulation has no cancellation counterpart, so cancellation is the weaker-guarded of the two equivalent terminal actions.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Harden `cancel()` toward parity with `resolve()`, and give users a bounded exit:

1. **Constrain owner cancellation.** Require a documented precondition before `cancel(false)` — e.g. forbid the no-refund sweep once `closeTime / resolutionTime` has passed (past that point the market should resolve, not sweep), or apply the same `resolutionDelay` cooldown to `cancel` so it cannot pre-empt a pending resolution. Reserve `cancel(false)` for genuinely void markets (never-graduated, no valid outcome), not for terminating live ones.
2. **Add a permissionless safety-valve cancel-with-refund.** After a long stall — e.g. `block.timestamp > resolutionTime + GRACE` with no resolution submitted — allow any user to trigger the refund path (`cancel(true)` semantics only, never the treasury sweep). This removes the owner as a single point of liveness failure without granting anyone the value-destroying no-refund branch.

Scope note: centralization hardening + design parity. If the team intends the owner to retain unconditional cancel authority, document that trust assumption explicitly (README + NatSpec) so integrators price it in.

Status

Acknowledged / Accepted Design — The owner-trust assumption is documented in [PR #86](#); no cancellation time gate was introduced.

[I-08] Cancelled-refund distribution gates the folded `seedFund` on `totalPool > 0` — a graduated market burned to `totalPool == 0` strands its retained seed backing

Field	Value
Severity	Informational
Source finding	I-13
GitHub issue	#77
Affected components	<code>Market._refundCancelledViaLedger (Market.sol:453-458 , seed distribution gated on <code>totalPoolForSeedSplit > 0</code>) ·</code> <code>Market._finalizeCancelledMarket /</code> <code>_foldPendingSeedIntoSeedFund (Market.sol:1354,1246) ·</code> <code>Market.burnCompleteSet (Market.sol:598-649) · recovery via</code> <code>Market.emergencyWithdraw (Market.sol:898-903)</code>
Confidence	High — refund math, <code>seedFund</code> lifecycle, the <code>totalPool == 0</code> reachability constraint, and the recovery path all traced end-to-end
Attack class	— (accounting completeness / residual-funds handling)

Description

What's wrong

When a graduated market is cancelled with `refund_ = true`, the seed collateral that was held back at graduation is folded into `seedFund` so it can be handed back to holders alongside their pool backing. But the per-user refund only pays out that `seedFund` when `totalPool > 0`, and it derives every outcome's backing from `pools[i]`. If the market's pools have been burned down to `totalPool == 0` (via complete-set burns) before the cancel, the folded `seedFund` is nonzero yet nothing is distributed: each refund computes to `0` and `redeem()` reverts `NothingToClaim`. The retained seed backing sits in the market's Ledger holdings, undistributed.

How (mechanism)

The seed is real, folded-in backing at a graduated cancel. Graduation zeroes `seedFund` (`Market.sol:1663`) but parks the retained ask-fund/dust in `pendingSeedPoolInflow`. On `cancel(true)`, `_finalizeCancelledMarket` calls `_foldPendingSeedIntoSeedFund` (`Market.sol:1354`), which restores it — `seedFund += pendingSeedPoolInflow` (`Market.sol:1246`) — so `seedFund` is nonzero going into the refund.

The refund gates seed distribution on live `totalPool` and reads backing from `pools[i]`:

```
// Market._refundCancelledViaLedger, :453-458
uint256 outcomeBacking = pools[i];
if (seedFundValue > 0 && totalPoolForSeedSplit > 0) {
    outcomeBacking += (seedFundValue * pools[i]) / totalPoolForSeedSplit;
}
uint256 share = (userBalance * outcomeBacking) / totalSupply;
```

When `totalPool == 0` : all `pools[i] == 0`, so the guard is false (seed term skipped) and `outcomeBacking == 0` → `share == 0` for every outcome → `totalRefund == 0` → `redeem()` reverts `NothingToClaim` (`Market.sol:428`). The folded `seedFund` is never routed anywhere on this path; it remains as residual `marketHoldings[market]` in the Ledger.

`totalPool == 0` is reachable only on a graduated market: complete-set burns decrement `pools[i]` and `totalPool` (`burnCompleteSet`, `Market.sol:644,649`), and are `onlyGraduated`. Post-graduation, `pools[i]` moves only via complete-set mint/burn.

Impact

- **Stranded value is protocol seed capital, not user principal.** The undistributed amount is the retained seed (ask-fund/dust) folded back into `seedFund` — market/ protocol funding, not depositor money.
- **No reachable stranded-user scenario.** `burnCompleteSet` reverts once any single pool would underflow (`pools[i] < outcomePoolDecrease`, `Market.sol:632`), so `totalPool` can only reach `0` by zeroing every pool together — i.e. with balanced pools. Because mint/burn move pool and supply at a fixed `n:1` ratio, zeroing a balanced pool also drives that outcome's supply to `~0`, so the directional holders have already exited (sold + burned) by the time `totalPool == 0`. A holder who refuses to sell keeps their side's pool non-zero and blocks the drain. So no refund-entitled token holder is left holding a position that returns `0`.
- **Recoverable, not permanently locked.** `emergencyWithdraw` (`onlyFactory` ← `onlyOwner`) sweeps residual `marketHoldings[market]` into the treasury bucket:

```
// Market.emergencyWithdraw, :898-903
uint256 ledgerResidual = ILedger(ledger).marketHoldings(address(this));
if (ledgerResidual != 0) ILedger(ledger).creditTreasury(ledgerResidual);
```

The seed is realized by the protocol rather than distributed. A permanent strand requires admin abandonment — a centralization/QA concern, not a code defect.

Net: a genuine distribution-formula gap (folded `seedFund` is silently undistributed when `totalPool == 0`), but bounded to admin-recoverable protocol seed with no reachable user-fund loss — hence Low/Informational rather than a fund-loss finding.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Close the residual on the finalization path instead of leaking it to a later `emergencyWithdraw` sweep. In `_finalizeCancelledMarket` (or the refund distribution), handle the `totalPool == 0 && seedFund > 0` case deterministically — pick one:

1. **Distribute by the cancellation supply snapshot.** Fall back to allocating the folded `seedFund` across outcomes by `cancelledOutcomeSupplyAtCancellation[i]` (the value already captured at cancel, `Market.sol:1356-1361`) when `totalPool == 0`, so any residual holder is still paid their pro-rata share of the seed:

```
// when totalPool == 0, back each outcome by its snapshot supply instead of pools[i]
uint256 denom = totalPool == 0 ? totalSnapshotSupply : totalPool;
uint256 weight = totalPool == 0 ? cancelledOutcomeSupplyAtCancellation[i] : pools[i];
uint256 outcomeBacking = pools[i]
    + (seedFundValue > 0 && denom > 0 ? seedFundValue * weight / denom : 0);
```

2. **Or sweep the residual to treasury on-path.** If no holder can be entitled when `totalPool == 0`, make that explicit: `creditTreasury(seedFund)` inside `_finalizeCancelledMarket` when `totalPool == 0`, so the accounting is closed at finalization rather than depending on a separate admin action.

Either removes the silent residual. Add a regression test: graduate → burn complete sets to `totalPool == 0` → `cancel(true)` → assert the folded `seedFund` is fully accounted (distributed or swept), and `marketHoldings[market]` nets to zero after finalization.

Scope note: accounting-completeness fix. No solvency change — the residual is already backed in `marketHoldings`; this just routes it deterministically.

Status

Fixed — Zero-pool cancellation now distributes retained seed backing using snapshotted outcome supplies, with a treasury fallback only when no holder supply exists. Merged in [PR #86](#).

[I-09] `marketBuy` / `marketSell` have no minimum-fill quantity floor — a partial fill succeeds silently, the only guard is the cost ceiling

Field	Value
Severity	Informational
Source finding	I-14
GitHub issue	#78
Affected components	<code>CLOB.marketBuy (CLOB.sol:317-348) · CLOB.marketSell (CLOB.sol:351-397) · _matchIncomingBuy (CLOB.sol:824) / _matchIncomingSell (CLOB.sol:948)</code>
Confidence	High — both market-order paths, their guards, and the absence of any <code>src/</code> caller verified end-to-end
Attack class	PartialFillValidation (benign leaf — terminal external order, no downstream consumer acting on the requested amount)

Description

What's wrong

A market order lets the caller cap what they *spend* but not what they *receive*. After matching, `marketBuy` reverts only on a zero fill or on exceeding the collateral ceiling — never on receiving fewer tokens than requested. A user who asks for 100 YES and finds only 1 available below their price ceiling gets 1 token, the call returns `receivedTokens = 1`, emits `MarketOrderExecuted`, and succeeds. `maxCollateralIn` is a cost ceiling, not a quantity floor, and the two are independent — you can receive 10% of the requested quantity while spending well under budget. `marketSell` mirrors this: its only guard is a proceeds floor (`minCollateralOut`), not a sold-quantity floor.

How (mechanism)

`marketBuy` runs the match then applies exactly two guards (`CLOB.sol:334-340`):

```
(filledAmount, spentCollateral, paidTakerFee) =
  _matchIncomingBuy(msg.sender, amount_, maxTick);
if (filledAmount == 0) revert InsufficientLiquidity(); // zero-fill only
uint256 spentWithFees = spentCollateral + paidTakerFee;
```

```
if (spentWithFees > maxCollateralIn_) revert SlippageExceeded(); // cost ceiling only
receivedTokens = filledAmount; // any 0 < fill <= amount_ passes
```

There is no `filledAmount == amount_` check and no `minFillAmount_` parameter. The match itself stops when the requested `amount_` is met, when liquidity below `maxTick` is exhausted, or when the remaining size drops below `minOrderSize` (`CLOB.sol:845`), so a partial fill simply means “acceptable-price liquidity ran out” — and the call returns successfully with whatever was filled. `marketSell` is symmetric: `filledAmount == 0` reverts, `receivedCollateral < minCollateralOut_` reverts, and any partial sell succeeds — unsold tokens are returned to the caller and its escrow basis released (`CLOB.sol:388-392`).

Impact

- **No fund loss.** The buyer pays only for what fills, at best-price-first fills up to `maxTick`, for `spentWithFees <= maxCollateralIn_`. No overpayment, no value extracted from any party. A partial fill is a re-try cost (submit again for the remainder), not a loss. The seller symmetrically receives `>= minCollateralOut_` for the portion sold and keeps the rest.
- **The fill is explicit, not silent on-chain.** `marketBuy` / `marketSell` return `(filledAmount, receivedTokens/soldTokens, spent/receivedCollateral)`. A correct caller reads the fill from the return values, not from `amount_`.
- **No in-protocol full-fill consumer.** No contract under `src/` calls `marketBuy` / `marketSell` — they are external entrypoints only. So the dangerous partial-fill variant (an in-protocol step that pipes the *requested* `amount_` into downstream logic assuming a full fill) does not exist here. The “integration pipes `amount_` and mismatches” concern is a hypothetical external integrator misusing the API — out of scope under the future-integration / speculation invalidators.
- **Residual UX gap (the reportable kernel).** On a prediction market, takers often have a target quantity (“500 YES to cover a position”). The API offers no all-or-nothing / minimum-quantity mode, so a caller who needs atomic quantity semantics must enforce it off-chain by inspecting the return values and reverting in their own wrapper. That is a missing convenience/safety parameter, not a protocol fund-safety defect.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Add an optional minimum-fill floor so callers can opt into quantity atomicity without forcing it on best-effort takers:

```
// marketBuy(uint256 amount_, uint256 maxCollateralIn_, uint256 minFillAmount_)
if (filledAmount < minFillAmount_) revert InsufficientFill(); // 0 default == today's behavior
// symmetric minSoldAmount_ on marketSell
```

A `minFillAmount_` of `0` preserves the current partial-fill semantics; a non-zero value gives all-or-nothing (`minFillAmount_ == amount_`) or minimum-quantity fills alongside the existing cost/proceeds guard. Document, in the `IOrderBook` NatSpec, that market orders may partially fill and that callers must read `filledAmount` / `receivedTokens` rather than assume `amount_` — so any future integrator does not inherit the silent-mismatch assumption.

Scope note: API-surface addition. No change to matching, accounting, or solvency; the default path is byte-for-byte the current behavior.

Status

Fixed — Minimum-fill overloads were added for market buys and sells while preserving backward compatibility. Merged in [PR #86](#).

[I-10] `getMarkets` computes `offset_ + limit_` before clamping — a large `limit_` with any nonzero offset reverts on overflow instead of returning the tail

Field	Value
Severity	Informational
Source finding	I-15
GitHub issue	#79
Affected components	<code>MarketFactory.getMarkets (MarketFactory.sol:382-385)</code>
Confidence	High — the checked-add overflow and the offset-0-only clamp behavior traced directly
Attack class	— (read-only robustness / pagination arithmetic)

Description

What's wrong

`getMarkets` is meant to support oversized `limit_` values by clamping the window to the array length — pass a huge `limit_` and get everything from `offset_` onward. It only half-works: the clamp is applied *after* computing `offset_ + limit_`, and on Solidity 0.8 that add is checked, so a near-maximum `limit_` overflows and reverts before the clamp can run. The overflow depends on `offset_`, so the function behaves inconsistently — `getMarkets(0, type(uint256).max)` returns all markets, but `getMarkets(1, type(uint256).max)` reverts, even though offset `1` is valid and the intended result is “every market after index 1”.

How (mechanism)

After the bounds guard (`MarketFactory.sol:378-380`) establishes `offset_ < markets.length`, the window is computed add-then-clamp (`MarketFactory.sol:382-385`):

```
uint256 end = offset_ + limit_;           // checked add – reverts (panic 0x11) on overflow
if (end > markets.length) {
    end = markets.length;
}
```

The clamp on the next line never executes when the add itself overflows. Overflow fires whenever `limit_ > type(uint256).max - offset_`:

- `offset_ == 0, limit_ == type(uint256).max` → `0 + max == max`, no overflow → clamps to `markets.length` → returns all markets. Works.
- `offset_ == 1, limit_ == type(uint256).max` → `1 + max` overflows → revert. The valid offset `1` is rejected purely by the pre-clamp arithmetic.

So the “support a large `limit_`” behavior the clamp exists to provide holds only for `offset_ == 0`; any `offset_ >= 1` with a near-max `limit_` reverts instead of returning the tail.

Impact

- **View-only, no security weight.** `getMarkets` is an external `view` pagination helper for frontends/indexers; no `src/` contract branches on it, and it moves no funds. A revert here locks nothing and breaks no protocol operation.

- **Trivially avoidable.** Only a near- `type(uint256).max limit_` combined with `offset_ >= 1` triggers it. Any caller passing a realistic `limit_` (a page size, or `markets.length`) never hits the overflow. It is a robustness/consistency defect, not a reachable operational failure.
- **Inconsistent contract surface (the reportable kernel).** The function advertises large- `limit_` support via the clamp but honors it only at `offset_ == 0`, so a caller that paginates with “give me everything after this offset” by passing a sentinel max `limit_` gets non-uniform behavior across offsets.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Clamp `limit_` to the remaining count *before* any addition. The `offset_ >= markets.length` guard already guarantees `offset_ < markets.length`, so `remaining` cannot underflow:

```
uint256 remaining = markets.length - offset_;           // safe: offset_ < length
uint256 count = limit_ < remaining ? limit_ : remaining; // clamp before adding
address[] memory result = new address[](count);
for (uint256 i = 0; i < count; i++) {
    result[i] = markets[offset_ + i];                  // offset_ + i <= length, no overflow
}
return result;
```

This makes `getMarkets` overflow-free for every `(offset_, limit_)` and returns the tail from any valid offset regardless of `limit_` magnitude. Add a regression test: `getMarkets(1, type(uint256).max)` returns `markets.length - 1` entries starting at index 1 (and `getMarkets(0, type(uint256).max)` still returns all).

Scope note: view-surface robustness fix. No accounting, matching, or solvency change.

Status

Fixed — Pagination now clamps the limit before addition. Merged in [PR #85](#).

[I-11] `graduateCallerRewardBps` is validated and reserved at creation but never paid — the entire graduate-caller bounty subsystem is dead code

Field	Value
Severity	Informational
Source finding	I-16
GitHub issue	#56
Affected components	<code>Market._seedOrderBookLiquidity (Market.sol:1561-1664),</code> <code>Market._graduate (Market.sol:486-497),</code> <code>MarketFactory._validateSeedLadderFeasibility</code> <code>(MarketFactory.sol:786-803)</code>
Confidence	High — the reward field is never read at runtime, the event field is hard-coded to <code>0</code> , and every companion helper is uncalled; all traced directly

Field	Value
Attack class	— (spec/implementation mismatch, dead code)

Description

What's wrong

The market config exposes a `graduateCallerRewardBps` field documented as a reward “paid to non-factory caller that triggers graduation,” and the factory reserves budget for it when it checks that a market can seed its order books. At runtime no reward is ever paid: the graduation path never reads the field, and the `MarketGraduated` event reports the reward as zero. An operator who sets `graduateCallerRewardBps > 0` silently delivers nothing to whoever triggers graduation, and the companion timing machinery that was meant to gate the bounty (`graduateCallerOverdueDelay`, `thresholdReachedAt`, `_isGraduationOverdue`) is fully orphaned — none of it is wired to any caller.

How (mechanism)

`graduate()` forwards the caller to `_graduate(msg.sender, ...)` (`Market.sol:477`), which forwards it to `_seedOrderBookLiquidity(triggeredBy, totalPoolValue)` (`Market.sol:496`). Inside `_seedOrderBookLiquidity`, the config is loaded (`cfg = orderBookSeedConfig`, `Market.sol:1574`) but `cfg.graduateCallerRewardBps` is **never read** — no `creditTreasury`, no `_creditUser`, no transfer. `triggeredBy` reaches only the event, whose reward argument is the literal `0` on both emission paths (`Market.sol:1568` and `Market.sol:1660-1662`).

The factory establishes the *intent* to pay: `_validateSeedLadderFeasibility` reduces the minimum viable seed fund by the reward bps before checking ladder constraints (`MarketFactory.sol:797-803`), so the config layer believes a slice of `seedFund` is carved out for the caller. Nothing carves it out.

The bounty's timing subsystem is dead in the same way. `_recordThresholdCrossing` (`Market.sol:1184-1194`) is the only writer of `thresholdReachedAt`, and it is never called from anywhere in `src/` — so `thresholdReachedAt` (a public storage var, `Market.sol:117`) is permanently `0`. `_isGraduationOverdue()` (`Market.sol:1196-1202`) reads that always-zero value and therefore always returns `false`, and it too has no callers. The `_graduate` docstring already records the design pivot that stranded all of this: “On Arbitrum gas is cheap, so the buyer that pushes `totalPool` over `graduationThreshold` graduates the market in the same tx; there is no keeper bounty.” (`Market.sol:481-483`).

Impact

- **Operators: a configured feature silently no-ops.** Any operator setting `graduateCallerRewardBps > 0` (a validated, documented field — `IMarket.sol:52-53`) expects the graduation caller to be paid. They are paid `0`. This is a spec-vs-implementation mismatch, not a fund loss: the un-paid slice simply stays in `seedFund` and flows to the order books / pending seed inflow as normal.
- **No liveness or DoS risk.** The bounty existed to incentivize an external caller to graduate a stuck market. Graduation cannot get stuck: a buy that crosses `graduationThreshold` graduates inline within `buyOnPhase1`, and `graduate()` is permissionless once the threshold is met (`Market.sol:488`). So markets graduate without any external incentive — the missing payment removes an incentive that the current architecture no longer needs.
- **Feasibility check is over-conservative in the safe direction.** Because `_validateSeedLadderFeasibility` deducts a reward that is never actually removed, market creation demands marginally more `seedFund` than runtime needs. This makes creation slightly stricter, never looser — no runtime revert, no under-seeding.
- **Dead code surface.** `graduateCallerRewardBps`, `graduateCallerOverdueDelay`, `thresholdReachedAt`, `_recordThresholdCrossing`, and `_isGraduationOverdue` are all unreachable/never-read, carrying storage cost (`thresholdReachedAt`) and misleading NatSpec (`IMarket.sol:52-59`) for behavior that does not exist.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Pick one direction and make the config honest.

1. **Remove the dead subsystem (recommended given the “no keeper bounty” pivot).** Delete `graduateCallerRewardBps`, `graduateCallerOverdueDelay`, `thresholdReachedAt`, `_recordThresholdCrossing`, and `_isGraduationOverdue`, and drop the reward-reduction branch in `_validateSeedLadderFeasibility` (`MarketFactory.sol:797-803`) and the reward field from `MarketGraduated`. This matches the documented design (`Market.sol:481-483`).
2. **Or implement the payment.** If the bounty is intended to ship, pay it in `_seedOrderBookLiquidity` before the ladder allocation, gated on `_isGraduationOverdue()` and skipped when `triggeredBy == factory`, and emit the real amount instead of `0`:

```
uint256 callerReward = 0;
if (triggeredBy != factory && cfg.graduateCallerRewardBps > 0 && _isGraduationOverdue()) {
    callerReward = (seedFundValue * cfg.graduateCallerRewardBps)
        / MarketConstants.BPS_DENOMINATOR;
    seedFundValue -= callerReward; // carve BEFORE bid/ask split
    ILedger(ledger).convertMarketHoldingsToDeposit(callerReward);
    ILedger(ledger).creditDeposit(triggeredBy, callerReward);
}
// ... existing bid/ask split over the reduced seedFundValue ...
emit MarketGraduated(triggeredBy, totalPoolAtGraduation, seedFundValue, callerReward);
```

This also requires wiring `_recordThresholdCrossing` into the `buyOnPhase1` threshold-crossing path so `thresholdReachedAt` is actually set and `_isGraduationOverdue()` can return `true`.

Scope note: no accounting or solvency change either way. Option 1 is a code-hygiene fix; Option 2 introduces a real payout and must be re-reviewed for the promo/basis interaction of crediting `triggeredBy`.

Status

Fixed — The factory rejects nonzero caller-reward configuration and removes the phantom feasibility deduction. Merged in [PR #85](#).

[I-12] Non-monotonic `seedFeeBpsPerTier` passes validation but diverges from the whole-buy runtime fee — the seed/rake safety model is misleading

Field	Value
Severity	Informational
Source finding	I-17
GitHub issue	#58
Affected components	<code>MarketFactory._validateSeedFeeConfig</code> <code>(MarketFactory.sol:605-636)</code> , <code>MarketFactory._validateSeedFeeRakeSafety</code> <code>(MarketFactory.sol:745-772)</code> , <code>Market._getSeedFeeBpsByPoolSize / Market.buyOnPhase1</code> <code>(Market.sol:1522-1539, Market.sol:251,277-280)</code>
Confidence	High — the missing bps-monotonicity check and the whole-buy runtime fee application both traced directly

Field	Value
Attack class	— (config validation vs runtime model mismatch)

Description

What's wrong

Seed fees are configured as a schedule of `(tierStart, feeBps)` tiers. The factory validates that the tier *starts* strictly increase but never checks that the *fee bps* are non-decreasing, so an operator can configure a fee schedule that spikes in a narrow middle band and drops again. The factory's seed/rake safety check models each tier's fee as applying only across that tier's own width, but at runtime a buy is charged a single fee — chosen from the pre-buy pool size — applied to the *entire* buy amount. When the schedule is non-monotonic, a large buy that starts at the spike tier pays that spike rate on its whole notional, generating far more seed fund than the width-weighted model assumed. The safety check therefore validates against a fee integral the runtime does not honor.

How (mechanism)

`_validateSeedFeeConfig` enforces strictly-increasing tier starts (`MarketFactory.sol:624-626`) and per-tier `bps <= BPS_DENOMINATOR` (`MarketFactory.sol:628-630`), but has **no check that** `seedFeeBpsPerTier[i] >= seedFeeBpsPerTier[i-1]`. A schedule like:

```
seedFeeTierStarts = [0, 500e6, 501e6];
seedFeeBpsPerTier = [0, 2000, 0]; // 20% spike in a 1-USDC-wide band
```

passes validation.

`_validateSeedFeeRakeSafety` models total seed as a **width-weighted integral** (`MarketFactory.sol:753-763`):

```
seedWeightedSum += uint256(cfg.seedFeeBpsPerTier[i]) * (tierEnd - tierStart);
```

For the schedule above this counts the 20% tier as `2000 * 1e6` — a negligible contribution, because the model assumes 20% is charged only across the 1-USDC band `[500e6, 501e6]`.

Runtime disagrees. `_getSeedFeeBpsByPoolSize(currentPoolSize)` selects the highest tier whose start `<= currentPoolSize` (`Market.sol:1529-1537`), and `buyOnPhase1` applies that one rate to the whole collateral: `poolContribution = collateralAmount * BPS / (BPS + seedFeeBps)`, `buyerSeed = collateralAmount - poolContribution` (`Market.sol:277-280`), with `seedFeeBps = _getSeedFeeBpsByPoolSize(totalPool)` read from the *pre-buy* pool (`Market.sol:251`). A buyer who buys a large amount when `totalPool == 500e6` is charged the full 20% on the entire buy — orders of magnitude more seed than the width-weighted `seedWeightedSum` predicted.

The safety check's own NatSpec already flags that `seedWeightedSum` is a *marginal-integral* upper bound (`MarketFactory.sol:726-734`); a non-monotonic schedule breaks that bound because the runtime is whole-buy, not marginal.

Impact

- **The seed/rake safety model loses fidelity, in the risk-increasing direction.** `_validateSeedFeeRakeSafety` exists to reject schedules where accumulated seed is too large relative to `rake`, because excess seed feeds the resolved complete-set payout ratio (the M-04 / `pendingSeedPoolInflow` surface). A non-monotonic schedule can make the modeled `seedWeightedSum` small while the runtime collects much more seed — so a config the check *passes* can behave like one it was meant to *reject*. The check's guarantee is silently weakened.
- **Admin-gated, so Informational.** `createMarket` is `onlyOwner` and the seed/rake schedule is a deliberate admin-chosen economic parameter (the code already accepts related multi-outcome model gaps as admin-owned risk — `MarketFactory.sol:739-744`). There is no permissionless path to install a malicious schedule, and no direct

fund-loss primitive here for a non-admin actor. The value is in preventing an operator from *unknowingly* shipping a schedule whose runtime seed accrual contradicts the safety model they trusted.

- **Confusing operator surface.** A schedule that looks bounded (20% only in a 1-USDC band) charges 20% on any large buy that starts at the band, which no operator reading the width-weighted model would expect.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Require `seedFeeBpsPerTier` to be non-decreasing in `_validateSeedFeeConfig`, so the runtime whole-buy fee never exceeds the width-weighted model's per-tier assumption:

```
// inside the existing loop in _validateSeedFeeConfig, alongside the tier-start check
if (i > 0 && bps < orderBookSeedConfig.seedFeeBpsPerTier[i - 1]) {
    revert IMarketFactory.SeedFeeBpsNotIncreasing(); // new error
}
```

With monotonic bps, the fee selected at any pool size is `<=` every tier reachable above it, so the whole-buy charge is bounded by the schedule the safety integral models. Add a regression test asserting `[0, 2000, 0]` (with increasing starts) now reverts, and that a monotonic schedule still passes.

Scope note: config-validation hardening only — aligns the creation-time safety model with runtime fee application. No change to matching, accounting, or solvency.

Status

Fixed — The factory now enforces non-decreasing seed-fee tiers. Merged in [PR #86](#).

[I-13] `processFinalizationBatch` has no per-book fault isolation or force-skip — a single reverting order cancel would brick finalization permanently

Field	Value
Severity	Informational
Source finding	I-18
GitHub issue	#80
Affected components	<code>Market.processFinalizationBatch (Market.sol:824-878), CLOB._cancelOrdersRange / _nextActiveOrderForFinalization (CLOB.sol:1330-1441)</code>
Confidence	High on the structural claims (cursor semantics, no skip path — traced directly); the “no current trigger” conclusion is an exhaustive ruling-out of the cancel-path revert candidates (see Analysis)
Attack class	— (liveness / fault-isolation architecture)

Description

What's wrong

A graduated market is wound down by `processFinalizationBatch`, which walks the outcome order books one at a time and cancels every resting order before the market can pay out. The walk is strictly sequential with a single stored cursor, and it has no way to isolate or skip a book — or an individual order — that fails to cancel. If any single order cancel ever reverts, the whole batch transaction reverts, the cursor never advances past the stuck book, and every retry hits the same order and reverts again. There is no `skipBook`, no timeout, and no alternative finalization path anywhere in the contracts, and the emergency-withdraw functions do not bypass the sequential walk. The market — and all funds behind it — would be frozen with no on-chain recovery. Today no order cancel can actually revert, so this is a latent single-point-of-failure, not a live DoS; the concern is that the design gives one reverting order the power to brick the entire market.

How (mechanism)

`processFinalizationBatch` loads the cursor once (`bookIndex = pendingFinalizationBookIndex`, `Market.sol:836`), runs the per-book `while` loop, and writes the cursor back **only after the loop exits** (`Market.sol:860`) — never per-book inside it. So a revert on book `i` rolls the whole transaction back and leaves `pendingFinalizationBookIndex == i` unchanged.

The cursor also cannot skip an individual poison order. Inside `_cancelOrdersRange`, the `fromOrderId` argument is used only as a return value; order *selection* is always `_nextActiveOrderForFinalization()` (`CLOB.sol:1348,1366`), which returns the next `active` order in strict price order (`CLOB.sol:1389-1441`). Once a given order is the next active one at its price, every call reaches it again; shrinking `maxOrdersPerBook_` cannot step over it.

No escape hatch exists. `Market.cancelAllOrders` is gated `onlyNotFinalizing` (`Market.sol:814`), so it is unavailable once finalization has begun. `emergencyWithdrawERC20` / `emergencyWithdrawOrderBookERC6909` (`Market.sol:881-927`) move residual tokens but do not advance the finalization cursor or complete `_finalizeResolvedMarket` / `_finalizeCancelledMarket`. `MarketFactory` only exposes the same `processFinalizationBatch` / `cancelAllOrders` entrypoints (`MarketFactory.sol:243,258`).

Analysis — why there is no current trigger

Every candidate revert in the per-order cancel path (`CLOB._cancelActiveOrder`, `CLOB.sol:1245-1292`) was ruled out against the current code:

- `releaseEscrowedSellBasis` → `UnknownEscrow`: ruled out. `escrowSellBasis` writes the escrow entry unconditionally (`Ledger.sol:965-971`), even for a zero-basis position; a tiny-but-nonzero basis that floors to zero reverts at *placement* (`Ledger.sol:953`, the [I-08] case). No escrow-less resting sell can exist to fail on cancel.
- `_safeTransferOutcome` → **receiver-hook revert**: ruled out. `OutcomeToken` is **ERC6909** (no `onReceived` callback); `transfer` reverts only on insufficient balance, and the CLOB custodies the escrowed tokens (`CLOB.sol:1695-1698`). A contract seller cannot inject a reverting hook.
- `releaseReservedBuy` → **mismatch / underflow**: ruled out in normal flow. It runs only when `reservedCollateralByOrderId > 0` (the reservation exists), `recipient == order.trader == r.user`, and the `totalReservedBuy` decrement mirrors the reservation-time increment.
- `onSeededBidRefund`: ruled out. Storage-add + event only, no Ledger call, no pause gate (`Market.sol:691-699`).
- **Per-order gas bomb**: ruled out. Cancel work is bounded by `MAX_COMPACT_STEPS_PER_CANCEL`; `maxOrdersPerBook_ = 1` always makes progress.

The per-book loop is also pause-exempt end-to-end (all three release calls omit `whenNotPaused`), so a pause does not revert the loop — the pause only bites the terminal `creditSaleProceeds`, which is [I-03]/[I-06].

Because no cancel revert is reachable, the “stuck forever” outcome has no PoC today. The finding is the *architecture*: one reverting order would be unrecoverable, and nothing structurally prevents a future change (a new callback, a new Ledger guard, a token swap away from ERC6909) from introducing exactly that revert.

Impact

- **Latent single-point-of-failure over the whole market.** A finalization that reverts on one order cannot advance; the market and every user balance behind it freeze with no on-chain recovery path. The blast radius is the entire market, triggered by one order — the classic shape a fault-isolation layer exists to prevent.
- **No current fund loss or DoS.** Every reachable order cancel succeeds today (see Analysis), so this is not exploitable now. It is graded Informational on that basis — the value is preventing a future regression from silently becoming a fund-freeze.
- **Fragile invariant to maintain by hand.** “No order cancel may ever revert” is an implicit, un-enforced invariant that every future change to the CLOB cancel path, the Ledger release functions, or the outcome-token standard must preserve, with no test or guard catching a violation.

Proof of Concept

No standalone proof of concept is included in the source write-up. The finding is supported by direct code-path and state-transition analysis.

Recommendation

Add per-book fault isolation and an admin force-skip so a single failing cancel degrades gracefully instead of bricking the market.

1. **Isolate each book with `try/catch`** in the loop so one book’s failure advances the cursor instead of reverting the batch, and record the skip:

```
try IOrderBook(orderBook).cancelOrdersRange(cursor, maxOrdersPerBook_, false)
  returns (uint256 nextCursor, bool done, uint256)
{
  pendingOrderCursorByBook[bookIndex] = nextCursor;
  if (done) { delete pendingOrderCursorByBook[bookIndex]; bookIndex++; }
  else break;
} catch {
  emit FinalizationBookSkipped(bookIndex, orderBook); // new event
  bookIndex++; // isolate, don't brick
}
booksProcessed++;
```

2. **Add an owner/factory `forceSkipFinalizationBook(uint256 bookIndex_)`** escape hatch (paired with `emergencyWithdrawOrderBookERC6909` to reclaim that book’s residual tokens) so a genuinely stuck book can be stepped over deliberately, with an emitted event for transparency.
3. **Persist the cursor per book** (write `pendingFinalizationBookIndex` inside the loop as each book completes) so partial progress survives even an un-isolated revert.

Scope note: liveness/robustness hardening only — no accounting or solvency change. Because there is no current trigger, this is a defense-in-depth fix, not an urgent patch.

Status

Fixed — Per-book fault isolation and an owner force-skip escape hatch were added and merged in [PR #86](#).

Conclusion

The report contains 24 standardized findings sourced from the complete Markdown write-ups. At publication, 18 findings are fixed and 6 are acknowledged or accepted design decisions. All remediations present in merged PRs are classified as fixed.